스킬 서포트 KVM & podman

가상화는 무엇인가?
실제가 아닌 것
real / virtual
컴퓨터 과학: 실제가 아닌 하드웨어 환경
가상화 기술 : 물리적 하드웨어 기능을 복제하여 운영시스템에 제공
클라우드 서비스를 위한 인프라
클라우드 컴퓨팅의 정의

클라우드 컴퓨팅은 소프트웨어를 비롯하여 인터넷 또는 전용 네트워크를 통해 프로비저닝한 서버, 스토리지 및 기타 컴퓨팅 리소스에 대한 액세스를 제공한다. 이러한 리소스는 일반적으로 사용자가 관리할 필요가 없고 리소스의 물리적 위치를 몰라도 된다는 점에서 위치 독립적이다. 사용자는 필요에 따라 IT 리소스를 구입하여 사용하고 사용량에 따라 서비스 요금을 지불하면 된다.

클라우드 컴퓨팅은 탄력적인 리소스 풀에서 IT 서비스를 프로비저닝하는 기능을 제공하는 가상화 기술을 기반으로 한다. 가상화 기술을 사용하면 하나의 물리적 시스템을 여러 가상 머신(Virtual Machine)으로 파티셔닝할 수 있다. 각 가상 머신은 별도의 물리적 리소스인 것처럼 다른 디바이스, 애플리케이션, 데이터 및 사용자와 독립적으로 상호 작용할 수 있다.

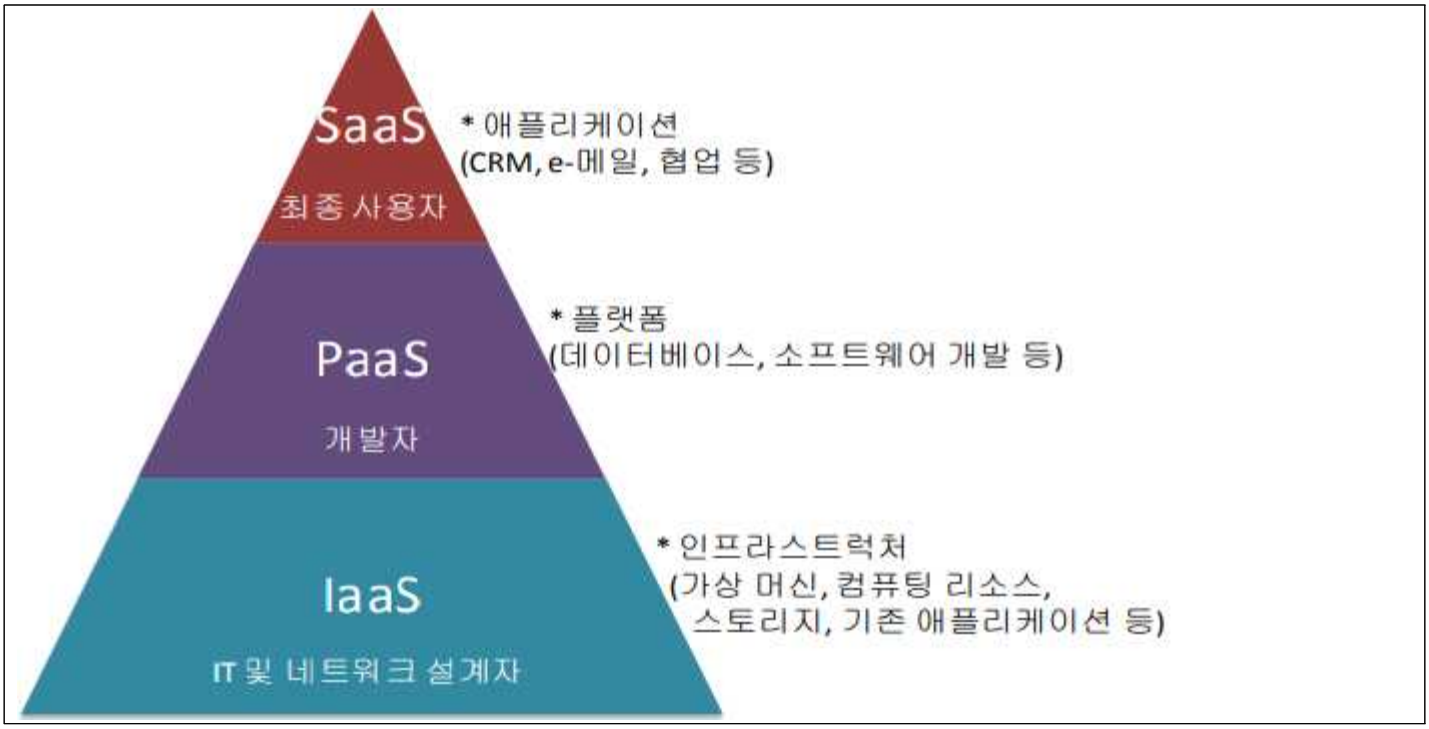
여러 가상 머신에서 단일 물리적 컴퓨터의 리소스를 공유하면서 서로 다른 운영 체제와 여러 애플리케이션을 실행할 수 있다. 각 가상 머신은 다른 가상 머신과 격리되어 있기 때문에 한 가상 머신에서 장애가 발생하더라도 다른 시스템에 영향을 미치지 않는다. 가상화 기술을 사용하면 하나의 시스템을 여러 가상 머신으로 파티셔닝할 수 있을 뿐만 아니라 여러 물리적 리소스를 단일 가상 리소스로 통합할 수 있다. 대표적인 예로 스토리지 가상화를 들 수 있다. 스토리지를 가상화하면 여러 네트워크 스토리지 리소스가 단일 스토리지 디바이스처럼 풀로 구성되므로 스토리지 리소스를 보다 쉽고 효율적으로 사용하고 관리할 수 있다.

가상화는 하이퍼바이저 소프트웨어를 통해 구현된다. 가상화 관리자라고도 하는 이 소프트웨어는 하드웨어와 운영 체제 사이에 위치하여 하드웨어의 구속으로부터 운영 체제와 애플리케이션을 분리한다. 하이퍼바이저는 운영 체제 및 애플리케이션에 필요한 만큼 프로세서와 기타 하드웨어 리소스(예: 메모리, 스토리지 시스템)를 할당한다. 클라우드 공급업체는 제공하는 서비스 유형(스토리지, 컴퓨팅, 대역폭, 활성 사용자 계정 등)과 관련된 사용량 측정 기능을 통해 리소스를 관리하고 최적화한다.

클라우드 컴퓨팅 서비스 모델
기업에서는 점점 증가하는 클라우드 기반 IT 서비스를 선택하여 사용하고 이를 여러 유형의 환경에 구축할 수 있다.
다음은 "ITaaS(IT as a Service)" 오퍼링의 가장 일반적인 세 가지 유형이다(그림 1 참조). SaaS(Software as a Service) – 인터넷을 통한 소프트웨어 애플리케이션을 액세스할 수 있도록 한다. 이러한 리소스를 구입하여 사용자의 컴퓨터나 디바이스에 설치하고 업데이트하고 관리하는 대신 웹 브라우저를 통해 액세스하여 사용할 수 있다. SaaS 공급업체는 클라우드에서 사용자를 위한 소프트웨어, 처리 능력 및 스토리지를 관리한다. 대부분의 SaaS 솔루션은 퍼블릭 클라우드(아래 설명 참조)에서 실행되며, 구독 기반 또는 무료 서비스로 제공된다. 대표적인 SaaS 애플리케이션의 예로는 Salesforce.com, Google Apps for Business, SAP SuccessFactors 등의 주문형 비즈니스 애플리케이션과 LinkedIn 과 Twitter 같은 무료 소셜 네트워킹 솔루션이 있다.

PaaS(Platform as a Service) – 통합 클라우드 환경에서 SaaS 애플리케이션을 개발, 테스트, 실행 및 관리하는 데 필요한 인프라스트럭처와 컴퓨팅 리소스를 제공한다. 인터넷에 연결할 수 있기만 하면 누구나 하드웨어, 운영 체제, 데이터베이스, 미들웨어 및 기타 소프트웨어를 소싱, 구입 및 관리하지 않고도 클라우드 기반 솔루션을 이용하고 개발할 수 있다. 대부분의 PaaS 공급업체에서는 기존 프로그래밍 툴보다 사용하기 쉬운 JavaScript, Adobe Flex, Flash 등의 툴을 제공한다. 따라서 사용자는 개발 환경을 소유하거나 관리하지 않고도 애플리케이션을 개발하고 구축하여 관리할 수 있다. 일부 널리 알려진 PaaS 공급업체로는 Google App Engine, Windows Azure, Salesforce의 Force.com 등이 있다.
IaaS(Infrastructure as a Service) – 사용자가 프로세싱, 스토리지, 네트워크 및 기타 기본 컴퓨팅 리소스를 프로비저닝할 수 있는 호스팅되는 IT 인프라스트럭처를 제공한다. 이러한 인프라 스트럭처는 IaaS 공급업체가 실행하고 관리하므로 사용자는

이를 기반으로 원하는 운영 체제 및 애플리케이션 소프트웨어를 실행할 수 있다. Amazon EC2(Elastic Compute Cloud), Verizon Terremark, Google Compute Engine 등이 IaaS 공급업체의 예이다.



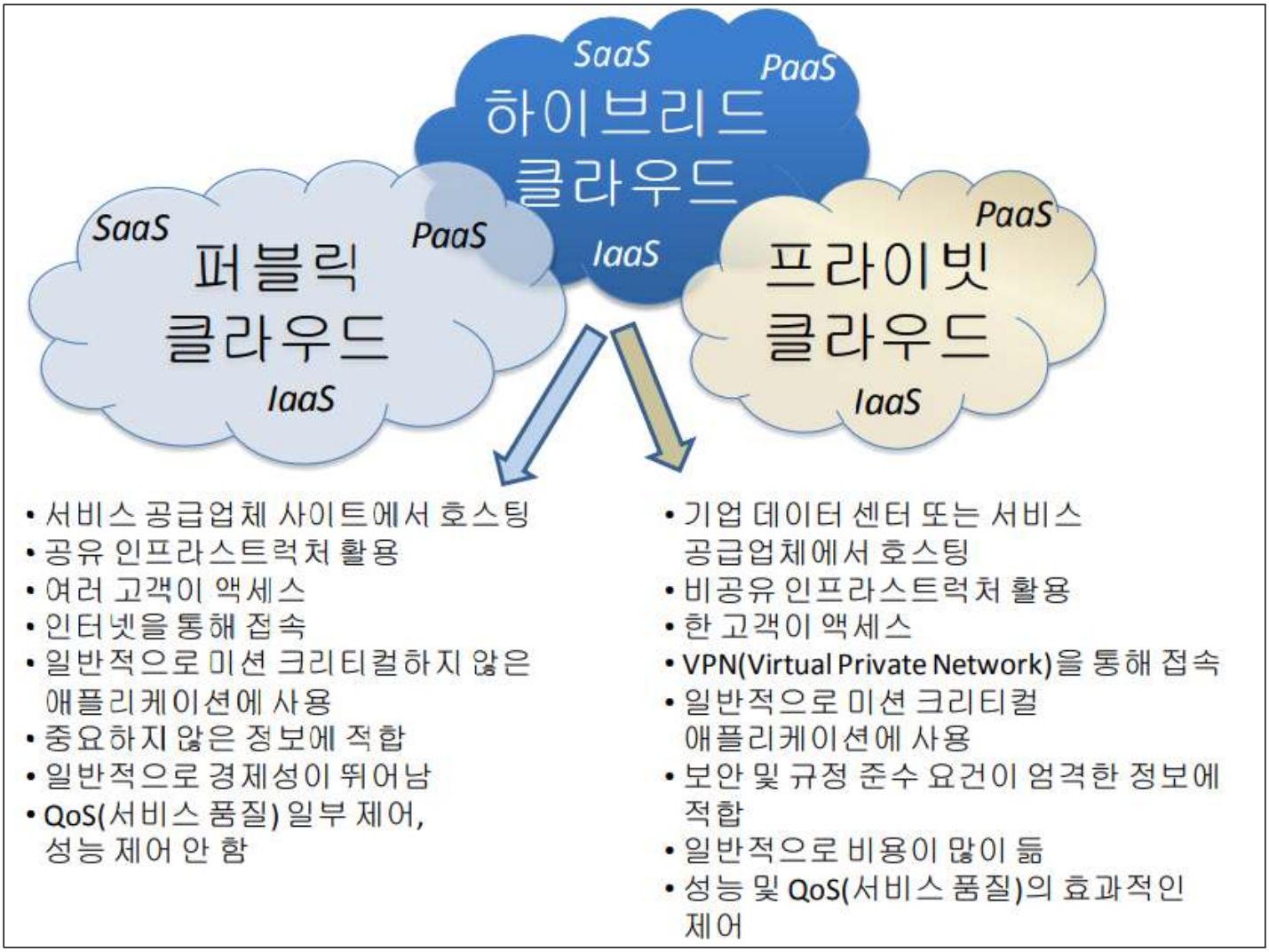
클라우드 컴퓨팅 IT 서비스 오퍼링

이러한 클라우드 서비스는 다음과 같은 유형의 환경에서 제공될 수 있다.

퍼블릭 클라우드 – 오늘날 가장 널리 알려져 있고 사용되고 있는 클라우드 환경이다. 퍼블릭 클라우드는 타사 클라우드 공급업체가 설계, 관리 및 유지하며, 구독자들 간에 서버, 스토리지, 네트워크 인프라스트럭처 및 애플리케이션을 공유한다. 이름에서 알 수 있듯이 누구나 서비스를 구입하여 이용할 수 있으며, 일반적으로 구독 가격 책정 모델이 적용된다. 대개 광고 수익으로 운영되는 무료 퍼블릭 클라우드 오퍼링도 제공된다. 일반적으로 퍼블릭 클라우드는 사용자가 원하는 리소스를 프로비저닝하는 셀프 서비스이다.

프라이빗 클라우드 – 특정 기업 적용으로 사용되며, 인프라스트럭처의 모든 리소스가 할당 기업 하나에 속할 있다. 프라이빗 클라우드 인프라스트럭처는 물리적으로 기업의 방화벽 내부 또는 타사 호스팅 공급업체의 사이트에 있을 수 있다. 기업에서는 자체 프라이빗 클라우드를 직접 관리하거나 타사 서비스 공급업체에게 관리를 맞길 수 있다. 어떤 방법으로 관리하든 상용 인프라스트럭처 하드웨어, 하이퍼바이저 및 관리 소프트웨어가 프라이빗 클라우드를 설계하고 관리하는 데 사용된다.

하이브리드 클라우드 – 퍼블릭 클라우드 리소스와 프라이빗 클라우드 리소스 모두로 구성되며, 내부 IT 직원과 타사 공급업체가 함께 소유하고 관리한다. IT 인프라스트럭처의 일부 구성 요소는 퍼블릭 클라우드에 있고, 나머지 구성 요소는 기업 내부에 있다. 예를 들어 특정 요구 사항에 따라 기업 내부의 프라이빗 클라우드에 데이터를 두고 퍼블릭 클라우드에서 분석 처리를 수행하거나, 그 반대로 활용할 수도 있다.



클라우드 컴퓨팅 환경

클라우드 서비스를 위한 요소 기술 소개

클라우드 컴퓨팅은 가상화, 분산 병렬 처리, 분산 데이터 저장 관리, 오픈 인터페이스(OpenAPI), 프로비저닝/자동화 도구, 모니터닝/미터링, 멀티테넌시(Multi- Tenancy), 보안, SLA 관리, 클리스터 관리 도구 기술의 집합체다. 오늘날 다양한 기술(제품)이 고객을 위해 준비되어 있다. 예를 들어 가상화를 지원하려면 CPU는 인텔 VT-x 나 AMD-V 를 반드시 지원해야 한다. 또한 Xen,KVM 같은 가상화 기술은 KVM같은 가상화 하이퍼바이저가 있어야 한다. 클라우드 컴퓨팅을 구현히는 주요 기술 요소는 아래의 표와 같다.

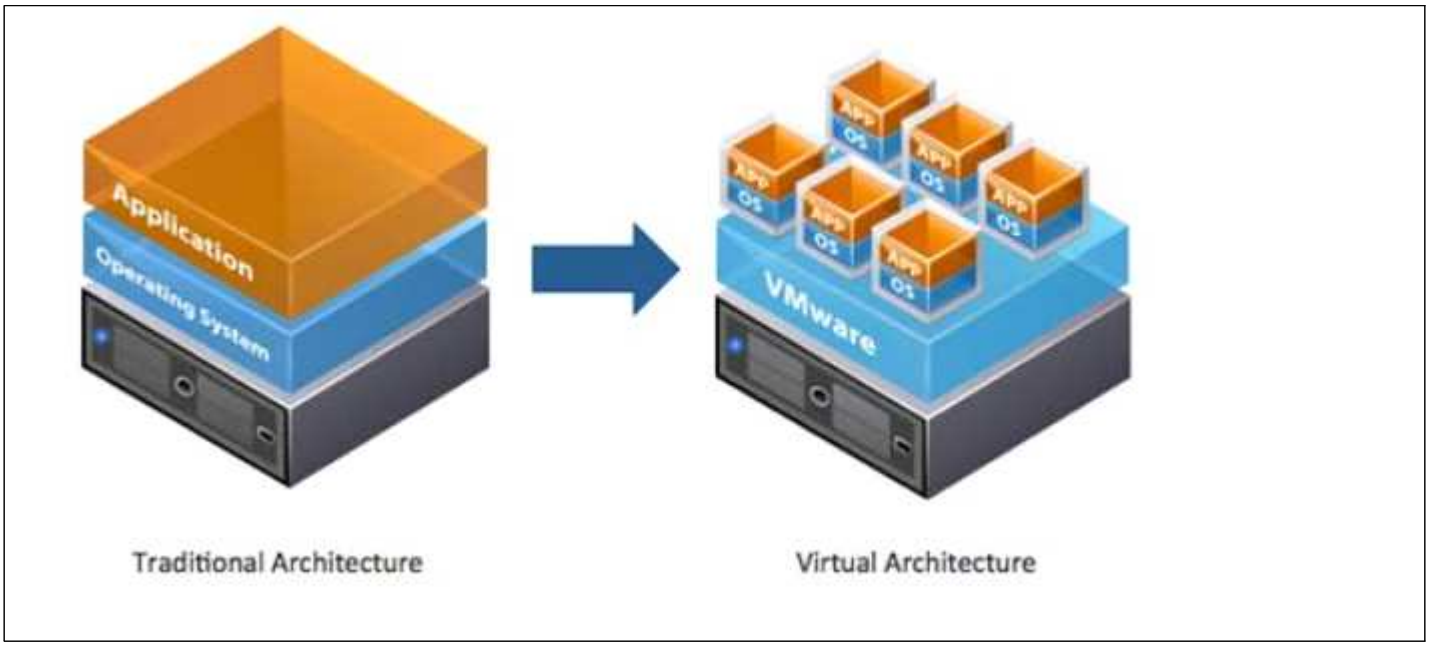
구분	설명
가상화	넓은의미로 컴퓨팅 자원의 추상화다. 주요 가상화 영역으로 서버 가상화, 데스크톱 가상화, 네트워크 가상화,스토리지 가상화가 있다. 가장 현장 수요가 높고 활성화된 분야는 서버 가상화 영역이다. 대표적인 솔루션으로 VMware, Xen, KVM이 있다. 가상화를 지원하는 CPU 기술로는 Intel-VT 나 AMD-V 가 었다
대규모 분산 병렬 처리	네트워크로 연결된 다수의 컴퓨터를 이용하여 거대한 작업을 작은 다수의 작업으로 나누어 병렬로 빠르게 처리하는 기술이다. 대표적위 기술로 하둡 맵리듀스, OpenMPI 가 있으며, 그래프 연산 작업을 위하여 프리젤을 사용한다.
분산 데이터 저장 관리	PB 이상의 대용량 데이터를 저장, 조회, 관리할 수 있는 기술이다. 파일 형식의 비정형 데이터를 관리하늒 분산 파일시스템과 정형 데이터를 실시간으로 저장 및 조회 할 수 있는 분산 데이터 관리 시스템으로 나뉜다. 분산 데이터 관리 시스템은 관계형 데이터베이스와 Key-Value 방식으로 데이터를 관리하는 NoSQL이 있다.
오픈 인터페이스	인터넷을 통한 서비스를 이용하고 서비스가 정보 공유를 지원하는 인터페이스 기술이다. 클라우드 기반 SaaS, PaaS에서 기존 서비스에 대한 확장 및 기능 변경에 적용할 수 있다. REST와 SOAP 기반 웹 서비스를 주요 기술로 활용한다.
프로비저닝 / 자동화 도구	서비스 제공자가 IT 자원을 할당, 배치, 배포 구성하는 작업을 자동화하여 사용자에게 제공하는 기술이다. 서비스 신청부터 자원 제공까지의 소프트웨어 라이프 사이클 관리를 실시간으로 할 수 있으며. 시스템 담당자의 개입이 필요 없는 자동화 기술이다.
모니터링/미터링	컴퓨팅 자원에 대한 사용량 수집, 헬스 상태를 감시한다. 자원 사용량수집이 목적이면 모

	니터링, 과금이 목적이면 미터링이라고 한다.
다중 공유 모델	하나의 인스턴스를 여러 사용자가 완젂히 분리된 형태로 사용하는 기술이다. SaaS 를 제공하는 데 핵심 요소다.
보안	민갑한 보안 정보를 외부 컴퓨팅 자원에 안전하게 보관하기 위한 기술이다. 방화벽, VPN, IPS 같은 네트워크 봉안 기술이 었다. 공인인증서 ,TLS 같은 데이터 암호화를 위한 기술도 필요하다. OVS 같은 가상머신을 위한 네트워크 기술이 있다.
SLA 관리	서비스 수준이라는 계량화된 형태의 품질 관리 기술이 필요하다.
관리 도구	클라우드 컴퓨팅을 구성하는 주요 시스템들에 대한 마스터 서버와 요소 서버를 관리하여 시스템 전체의 고가용성을 확보하는 기술이다. 단순한 모니터링뿐만 아니라 활용 정도에 따라 동적인 자원 할당 및 스케쥴링 기능을 제공한다.

가상화의 이해

최근 기업의 IT 환경에서 가장 주목받고 있는 기술은 가상화다. 지난 시간 동안 기업에서 운영하는 서버는 150배 이상 증가했고, 운영 인력 비용도 크게 늘었으며에너지 소비 문제까지 다양한 효율 문제가 대두됨에 따라, 더 이상 서버 수를 늘리는 방법으로는 혁신을 이어갈 수 없는 현실에 이르렀다. 혁신을 위한 새로운 기술이 필요해진 상황이 되었다. 이에 기업들이 떠올린 대안은 가상화 기술이다. 가상화는 기업이 직면한 다양한 문제를 해결하고, 효율성을 높이기 위한 대안으로 최적이었다. 그리고 가상화의 필요성은 최근의 경제 위기로 인해 더 주목받고 있다. 가상화를 통한 물리적인 서버 수의 감소, 더 쉽고 빠른 관리, 더 신뢰성있는 인프라의 구성에 있어 가장 현실적인 대안이다.

가상화란, IT 자원을 논리적인 자원 개념으로 재구성하여 자원의 물리적인 경계를 극복하는 개념이다. 각각의 시스템과 디스크 등의 자원을 물리적인 경계 단위로 별도로 관리하는 값이 아니라, 논리적으로 하나의 자원 형태로 통합함으로써, 더 유연한 분배와 사용이 가능하여 자원 효율성 최적화 및 관리 효율성을 극대화할 수 있다.



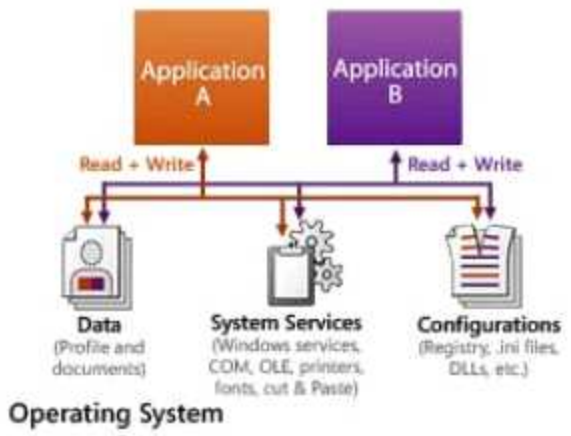
가상화의 역사

가상화는 1960년대에 시분할기법이 등장하면서부터 이미 예견되었다. 옥스퍼드대학의 교수이자 The Programming Research Group의 리더였던 크리스토퍼 스트레이치는 자신의 논문인 'Time Sharing in Large Fast Computers'에서 최초로 '시분할'이라는 단어를 사용하면서 하나의 머신에 다중 접속하여 서로 다른 프로그램을 작성하는 방법을 소개했다. 이에 멀티 프로그래밍이 가능해졌지만 당시에는 이런 시스템의 사용료가 너무 비쌌다.

그 덕에 세상의 빛을 보게 된 OS가 벨연구소의 유닉스다. 실제 가상화 시스템으로 운영된 최초의 시스템으로는 IBM 7044가 있으며, 맨체스터대학의 Atlas 프로젝트는 MIT가 개발한 IBM 704에 CTSS(Compatible Time Sharing System), 디맨드 페이징(demand paging), 슈퍼 바이저 호출(supervisor calls) 기능의 구현을 주도했다.

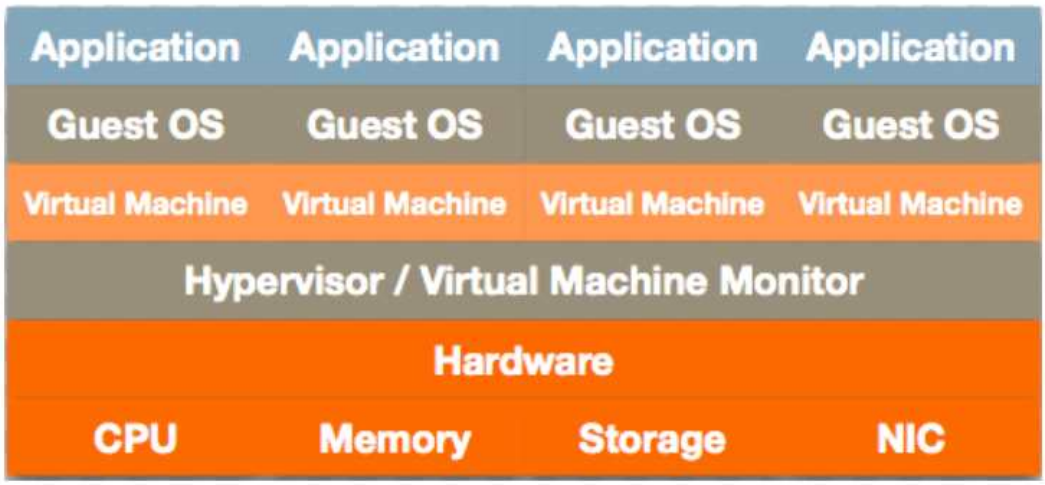
애플리케이션 가상화

또 다른 초기 가상화에는 P-code (또는 가상 코드 pseudo-code)머신도 있다. P-code는 물리적 하드웨어가 아닌 가상 머신에서 실행되는 머신 언어이다. P-code 는 1970 년대 초반, University of California, San Diego (UCSD) Pascal 시스템으로 유명할 진 것으로서, Pascal 프로그램을 P-code로 컴파일 한 다음, P-code가상 머신에서 실행한다. 따라서 P-code 프로그램은 뛰어난 이식성을 갖추게 되고, P-code 가상 머신을 사용할 수 있는 곳 어디에서나 실행된다.
Java Virtual Machine (JVM)자바 언어는 가상 머신에 P-code 모델을 따랐다. 따라서, JVM으로 간단히 이식하기만 해도 무한한 아키텍처에 자바 프로그램들을 제공할 수 있었다. C 언어의 조상격인 1969년대의 Basic Combined Programming Language (BCPL)에도 같은 개념이 적용되었다. 이 때에는 컴파일러가 BCPL 코드를 O-code 라고 하는 중간 머신 코드로 컴파일 한다. 두 번째 단계로, O-code는 대상(target) 머신의 네이티브 언어로 컴파일 되었다. 이 모델은 혁대적인 컴파일러에 의해 사용되어, 새로운 대상 아키텍처로 컴파일러를 이식할 때 유연하다. (중간 언어(intermediate language)에의한 프롞트엔드와 백엔드를 구별한다.



하드웨어 가상화

IBM 은 System/360™ Model 67 메인프레임을 개발하면서, 1960년대에 이미 가상화의 중요성을 인식했다. Model 67 은 Virtual Machine Monitor(VMM)을 통한 모든 하드웨어 인터페이스들을 가상화했다. 초기 컴퓨팅 시절에, 이 OS는 수퍼바이저(supervisor)로 불렸다. 다른 OS상에서 OS들을 실행할 수 있는 기능을 겸비하면서 하이퍼바이저(hypervisor)라는 이름도 얻게 되었다. (이 용어는 1970 년대에 만들어졌다. VMM은 기반 하드웨어에서 직접 실행되기도 하고, 다중의 가상 머신(VM)들도 실행할 수 있다. 각각 VM 은 고유의 OS 인스턴스를 실행할 수 있다. 초기에는 이것을 CMS(또는 Conversational Monitor System)라고 불렀다. VM 은 계속 성장했으며, 요즘에는 System z9™ 메인프레임에서 실행된다. System/360 계열 제품들과도 백워드 호환이 된다.



데스크탑 가상화

최근 들어 그 유용성에 대한 검토의 필요성이 높아진 것은 데스크톱 애플리케이션이나 데스크톱의 가상화다. 이들은 애플리케이션 스트리밍이나 서버 기반 컴퓨팅이라 불린다. 데스크톱 가상화의 주된 목적은 클라이언트 운용 관리성의 향상과 시큐리티의 향상이다. 그러나 아직 데스크톱 가상화는 새로운 분야이므로 다양한 단점도 존재한다. 마지막으로 서비스의 가상화가 있다. 이것은 사용자가 SaaS에의해 많은물리 자원을 사외 서비스의 제공자로부터 얻는 방식이다. 사용자는 서버나 스토리지, 데이터기반이나 애플리케이션을 보유하고 있지 않아도 네트워크에 접속할 수 있는 클라이언트만으로 애플리케이션을 이용할 수 있다.

가상화에 대한 제안

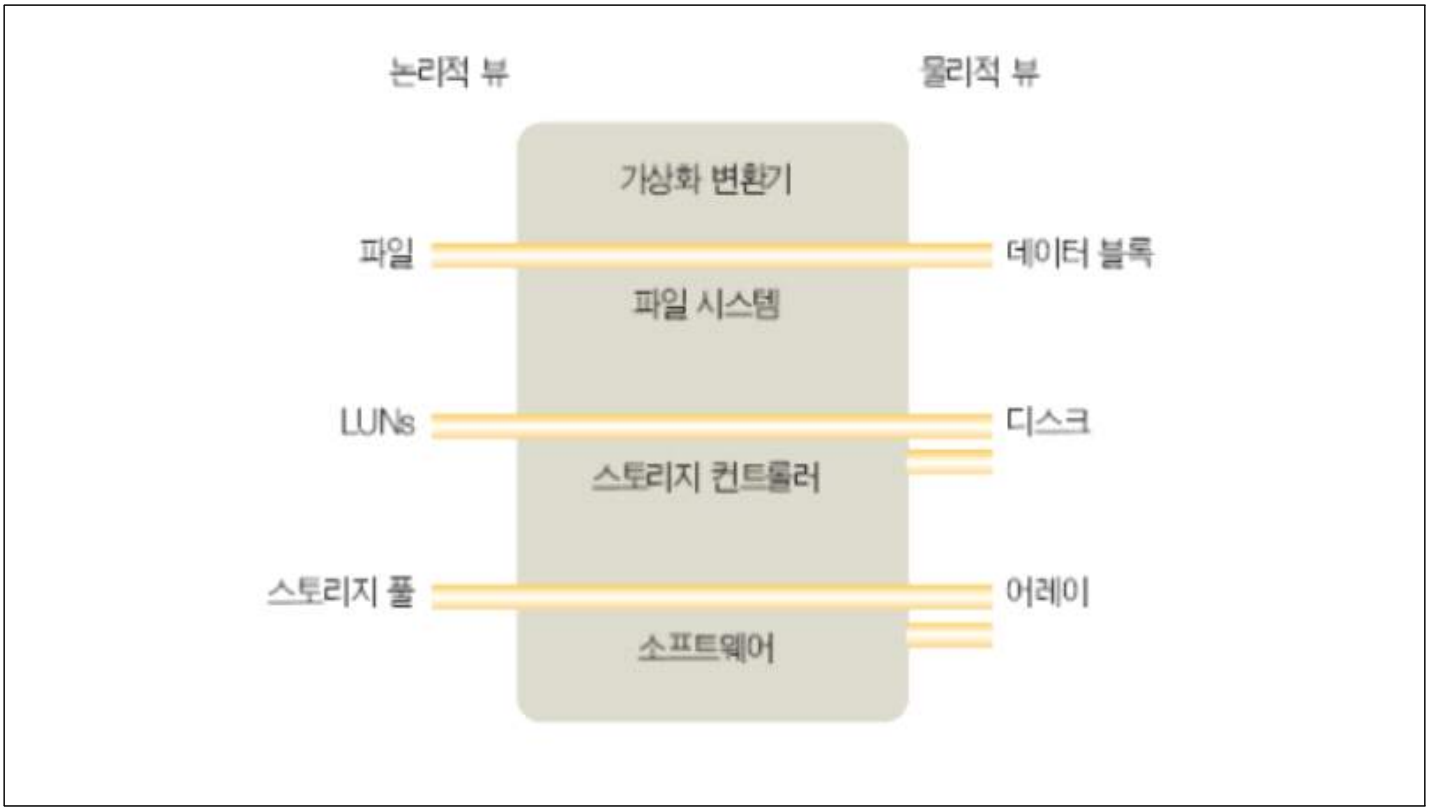
스토리지의 가상화나 서버의 가상화 등은 이미 활성화되어 북미에서는 약 60%의 기업이 서버의 가상화를 도입했다. 이번에 설명한 각 IT 스택의 가상화에는 엄밀하게 말할 가상화가 아니라고 말할 수 있는 것도 포함돼 있다. 그러나 사용자나 개발자를 다양한 자원의 제한으로부터 해방시켜준다는미에서 가상화란 표현을 써 보았다. IT 가 비즈니스를 실행하기 위한 도구인 이상, 사용자에게 보다 유연하고 이용하기 쉬운 것이 되지 않으면 안 된다. 가상화는 그것을 위한 효율적인 수단이라 여겨진다. 그러므로 사용자는 가상화의 단점과 이점을 충분히 고려한 다음 적극적으로 채용 검토를 해야 한다.

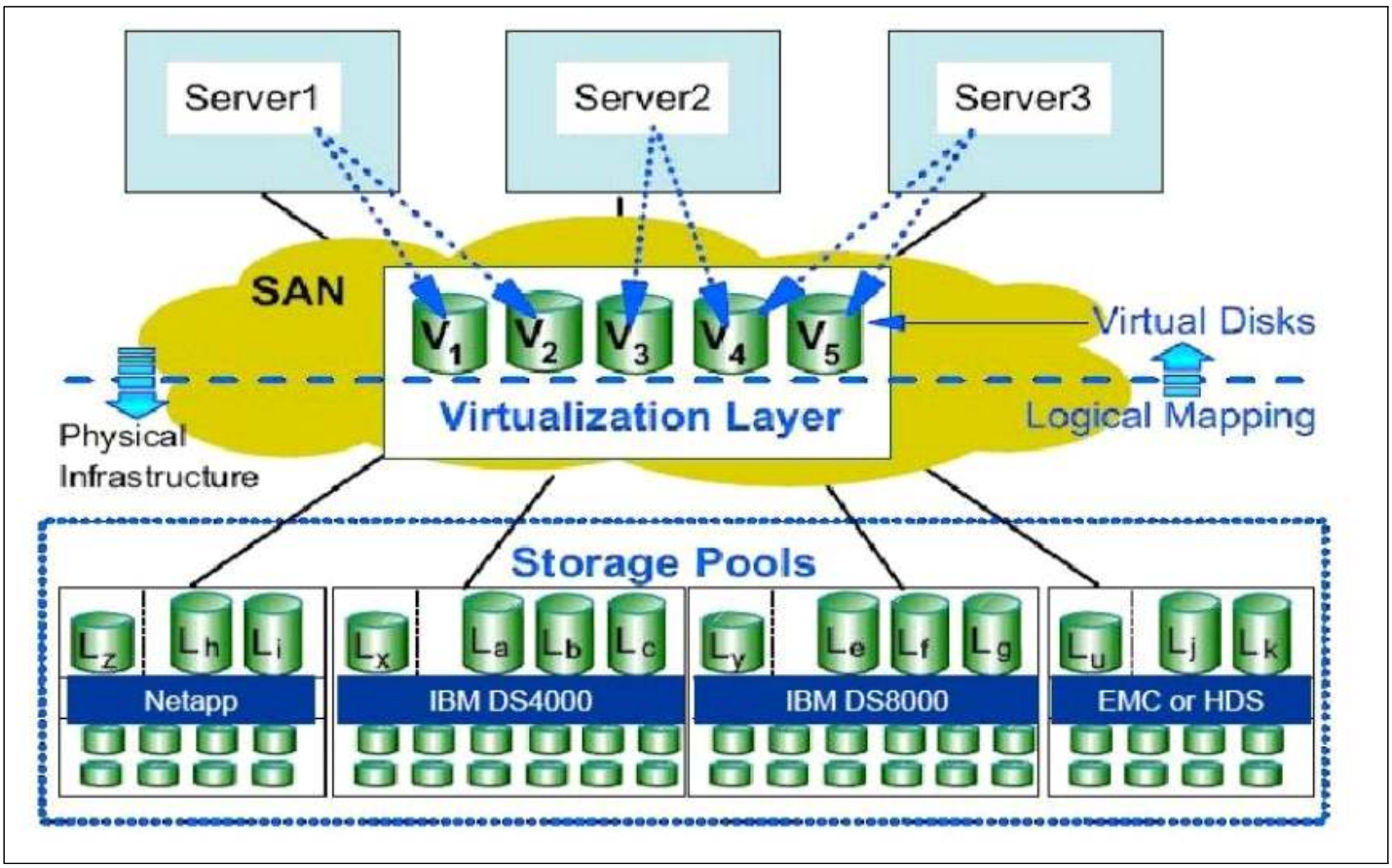
네트워크 가상화

네트워크 가상화는 하드웨어 네트워크 리소스 및 소프트웨어 네트워크 리소스를 단일 관리 단위로 결합하는 프로세스다. 네트워크 가상화의 목적은 시스템과 사용자에게 네트워크 리소스의 효율적이고 제어된 보안 공유를 제공하는 것이며 네트워크 가상화의 최종 결과물은 가상 네트워크다. 가상 네트워크는 두 가지 광범위한 유형인 외부와 내부로분류된다. 외부 가상 네트워크는 소프트웨어에서 단일 엔티티로 관리하는 여러 로컬 네트워크로 구성된다. 클래식 외부 가상 네트워크의 블록은 스위치 하드웨어 및 VLAN 소프트웨어 기술이다. 외부 가상 네트워크의 예로 대규모 회사 네트워크와 데이터 센터가 있다. 내부 가상 네트워크는 하나 이상의의사 네트워크 인터페이스에 구성된 가상 시스템 또는 영역을 사용하는 시스템 한 개로 구성된다. 이러한 컨테이너는 동일한 로컬 네트워크에 있는 것처럼 서로 통신하여 단일 호스트에 가상 네트워크를 제공할 수 있다. 가상 네트워크의 빌딩 블록은 VNIC(가상 네트워크 인터페이스 카드 또는 가상 NIC) 및 가상 스위치다. Oracle Solaris 네트워크 가상화는 내부 가상 네트워크 솔루션을 제공한다. 네트워킹 리소스를 결합하여 내부 및 외부 가상 네트워크를 모두 구성할 수 있다. 예를 들어, 내부 가상 네트워크가 있는 개별 시스템을 대규모 외부 가상 네트워크의 일부인 LAN 으로 구성할 수 있다.

스토리지 가상화

서버단의 가상화는 운영체계, 파일 시스템, 볼륨 관리 등에서 이미 존재하고 있다. 예를 들어 윈도우를 사용하는 문서 폴더는 하드디스크 드라이브에 저장된 데이터 블록의 가상화의 표현이라 할 수 있다. 윈도우 파일 시스템은 가상화 변환기(translator)'라 볼 수 있는 것이다. 폴더라든가 파일 목록은 바로 물리적인 데이터 블록의 논리적인 뷰인 것이다. 변환기의 기능은 다양한 메커니즘을 통할 물리적인 데이터 블록 저장 형태를 논리적인 뷰로 볼 수 있도록 하는 것에 있다. 이와 마찬가지로 RAID 컨트롤러도 역시 디스크에 물리적으로 분산 저장된 데이터 블록들을 논리적인 관점에서 LUN 들을 재표현했다고 할 수 있다. 여기서 만들어진 LUN 들은 정적인 방식이나 동적인 방식으로 로드밸런싱을 할 수 있다. 정적인 방식의 로드 밸런싱은 LUN 들이 특정 컨트롤러 경로에 할당됐을 때 관리하는 방식이다. 예를 들면 높은 I/O 를 요구하는 스토리지인 경우인 LUN A 를 컨트롤러 1에, 그리고 LUN B 를 컨트롤러 2에 할당하고 정적으로 컨트롤러 간의 로드 밸런싱을 하는 이다. 동적인 로드 밸런싱은 여러 가지 옵션으로 제공될 수 있는데 순환 순서 방식(round-robin), 최소 I/O, 최소 대역폭 등을 고려해 여러 개의 LUN 을 동일한 컨트롤러의 여러 포트에 할당하는 방식이다. 예를 들면 LUN A 와 LUN B 를 컨트롤러 1에 할당할 I/O는 포트 1 과 포트 2 사이에서 발생하도록 조정하는 방식이다. 이러한 방식들은 현재 많은 스토리지 시스템들이 혼용해 사용하는 기술이다. 가상화 소프트웨어는 여러 어레이 시스템에 분산 저장된 데이터 블록을 논리적인 뷰, 가상화 스토리지 풀로 표현할 수 있도록 해준다. 즉, 온라인 환경의 물리적인 장벽을 허물고 하나의 가상화 풀을 만들어 준다. 이것은 스토리지 자산의 효율성을 높여주며 이기종으로 구성된 호스트/스토리지 환경에서 미러링, 스냅샷을 통할 스토리지 관리상의 이점을 극대화할 수 있다.

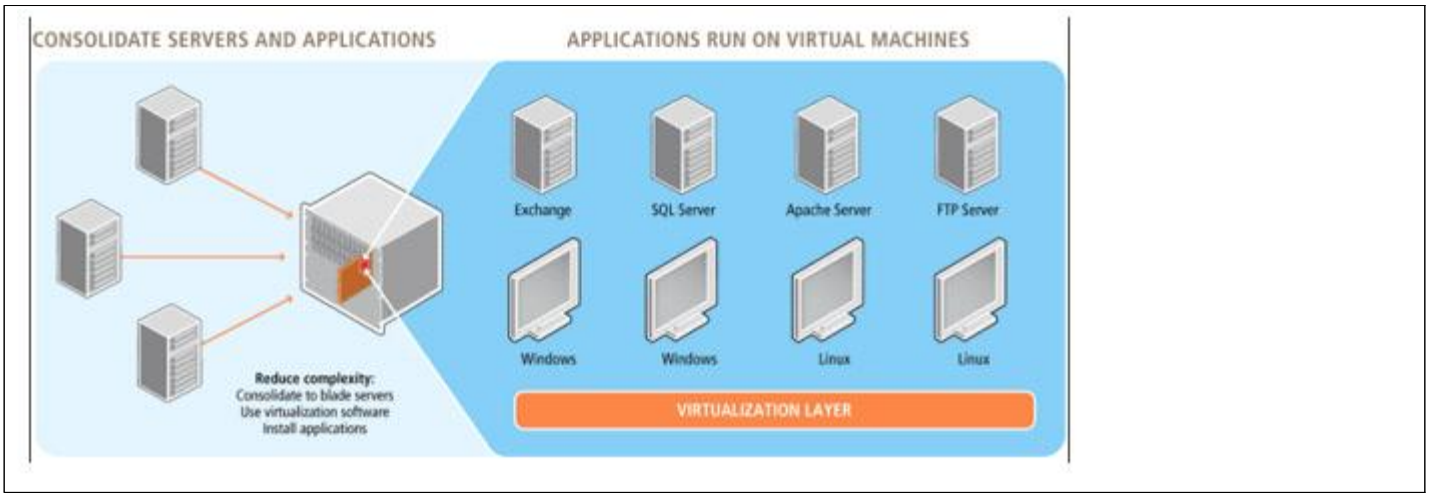

논리적 뷰
물리적 뷰
가상화 변환기
파일
데이터 블록
파일 시스템
LUNs
디스크
스토리지 컨트롤러
스토리지 풀
어레이
소프트웨어


Server1
Server2
Server3
SAN
V1
V2
V3
V4
V5
Virtual Disks
Virtualization Layer
Logical Mapping
Physical
Infrastructure
Storage Pools
Lz
Lh
Li
Lx
La
Lb
Lc
Ly
Le
Lf
Lg
Lu
Lj
Lk
Netapp
IBM DS4000
IBM DS8000
EMC or HDS

서버 가상화

가상화 범위를 서버 가상화로 좁혀 보면, 크게 세 가지 형태를 볼 수 있다. 물리자원을 단순히 덧셈하듯이 조합하여 자원을 할당하는 하드웨어 분할(Hardware Partitioning), 서버의 펌웨어나 소프트웨어 단에서 가상화 자원을 직접 관리하는 베어메탈 하이퍼바이저(Bare-Matal Hypervisor), 호스트 OS 위에 하이퍼바이저가 올라가는 형태의 호스티드 하이퍼바이저(Hosted Hypervisor)가 그것이다.

하드웨어 분할은 서버 자원을 물리적인 단위에서 분리하여 사용한다. 가장 대표적으로, 다수의 메인보드가 장착되어 있거나, 블레이드 서버가 랙에 조합되어 하나의 시스템을 구성하는 분산 시스템의 경우 메인보드나 블레이드 서버의 할당 수를 조절하여 하나의 서버에서 완전히 독립된 다수의 가상 서버를 만들어낼 수 있다. 하지만 자원 할당의 정교함이 떨어지고, 가용성과 효율성 측면에서도 큰 이득을 얻기 힘들다. 이는 IBM의 구형 메인프레임급 서버에서 주로 찾아볼 수 있던 방법이며, 현재는 거의 사용되지 않는다. 현재 가장 흔히 본 수 있는 가상화 방법은 호스티드 하이퍼바이저이다. 서버 위에 호스트 레벨의 OS 가 있고, 가상화 관리자인 하이퍼바이저는 OS에서 별도의 프로그램이나 서비스 형태로 구동된다. 이 경우 서버의 자원을 공유하기 위할 호스트 OS의 지원을 받게 된다. 호스트 OS가 인식하는 모든 자원을 그대로 사용할 수 있어 가용성, 유연성이 뛰어나다. 하지만 때에 따라 호스트 OS에 가상 시스템들의 특성이 종속되는 경우가 발생하며, 성능의 효율성 또한 다소 떨어지는 경우가 있다.



가상화의 종류

요즘 가상화 기술이 주목을 받고있다. 그중에서도 서버의 가상화는 많은 회사들이 안고있는 하드웨어의 관리, 재난에의한 시스템의 신속한 복구등 여러가지 문제를 해결해 줄 방법으로 각광 받고있다. 이런 서버의 가상화는 하이퍼바이저, 가상 OS, VMM(Virtual Machine Monitor=가상머신에 CPU 나 기억장치 등의 리소스를 가상화하여 제공하는 소프트웨어,가상화 레이어라고도 함)라고도 불리며 크게 두가지 "Type1 (네이티브, 베어메탈 방식)"과 "Type2 (호스트형 방식)"으로 나눌 수 있다.

Type2 가상화

일반적으로 호스트(Hosted)형 가상화

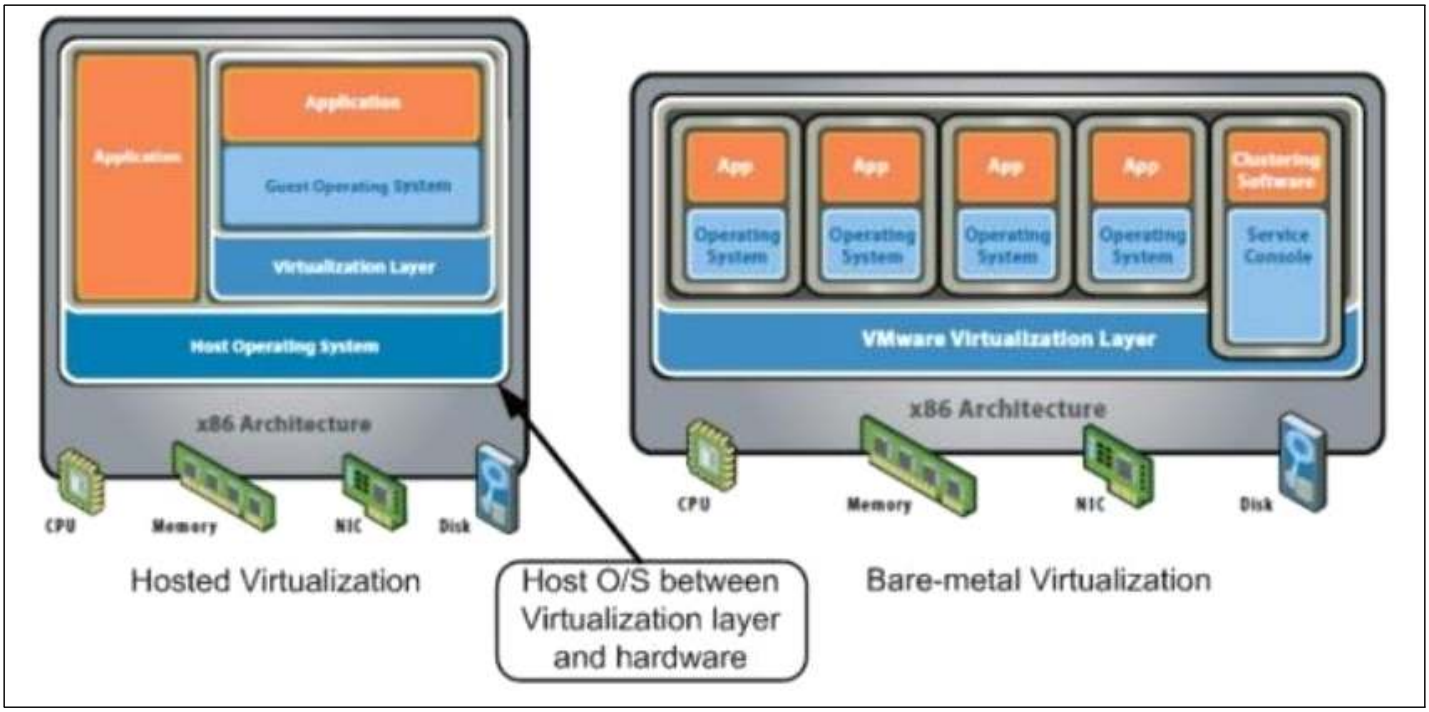
호스트형 가상화는 물리 컴퓨터상의 호스트 OS 위에 VMM(Virtual Machine Monitor)가 설치되며 VMM 위에 게스트 OS 를 동작시키는 방법이다. 호스트형 가상화는 물리 컴퓨터의 하드웨어를 에뮬레이트 하기 때문에 오버헤드는 크지만, 게스트 OS의 종류에 그다지 제약이 없다. 그렇기 때문에 Windows에서 FreeBSD 까지 다양한 게스트 OS 를 동작시킬 수 있고, 물리 컴퓨터 역시 데스크톱뿐만이 아니라, 노트북에서도 동작한다. 이런 호스트형 가상화 소프트웨어에는 VMware의 VMware Workstation, VMware Server, VMware Player, 오라클의 VirtualBox, Parallels의 Parallels Workstation 등이 있다. 이 호스트형 가상화는 무엇보다도 손쉽게 도입이 가능하다는 점이라 생각한다.

Type1 가상화

하이퍼바이저(Hypervisor)형 가상화

하이퍼바이저형 가상화는, VMM 을 물리 컴퓨터의 하드웨어상에 직접 동작을 시키는 방법으로 호스트 OS 가 필요없다. 호스트 OS에 할당할 리소스가 필요없기에 호스트형 가상화에 비해 오버헤드가 적고 물리 컴퓨터 리소스의 관리가 유연한게 특징이다. 하지만 자체적인 관리기능이 없기에 별도의 관리콘솔(내지는 관리 컴퓨터,RHEV-M)가 필요하다. 하이퍼바이

저형 가상화 소프트웨어에는 VMware의 ESX/ESX i Server, Citrix의 XenServer, 마이크로소프트의 Hyper-VParallels의 Parallels Server 등이 있다.



전 가상화

전체 가상화(네이티브 가상화) 또한 가상화 방식이다. 이 모델은 게스트(guest) OS 들과 네이티브 하드웨어 사이를 중재(mediate)하는 가상 머신을 사용한다. "중재(Mediate)"는 핵심적인 단어이다. VMM 이 게스트 OS 와 베어 하드웨어 사이를 중재하기 때문이다. 보호를 받고 있는 특정 명령어들은 하이퍼바이저 내에서 트랩핑(trap) 및 핸들되어야 한다. 기반 하드웨어는 OS 가 소유한 것이 아닌, 하이퍼바이저를 통해서 공유되기 때문이다. 전체 가상화는 하드웨어에뮬레이션 보다는 빠르지만, 하이퍼바이저 중재 때문에 실제 하드웨어 보다는 성능이 낮다. 전체 가상화의 가장 큰 장점은 OS를 수정하지 않고 실행될 수 있다는 점이다. 유일한 제한 사항은 OS 가 기반 하드웨어를 지원해야 한다는 것이다.

반 가상화

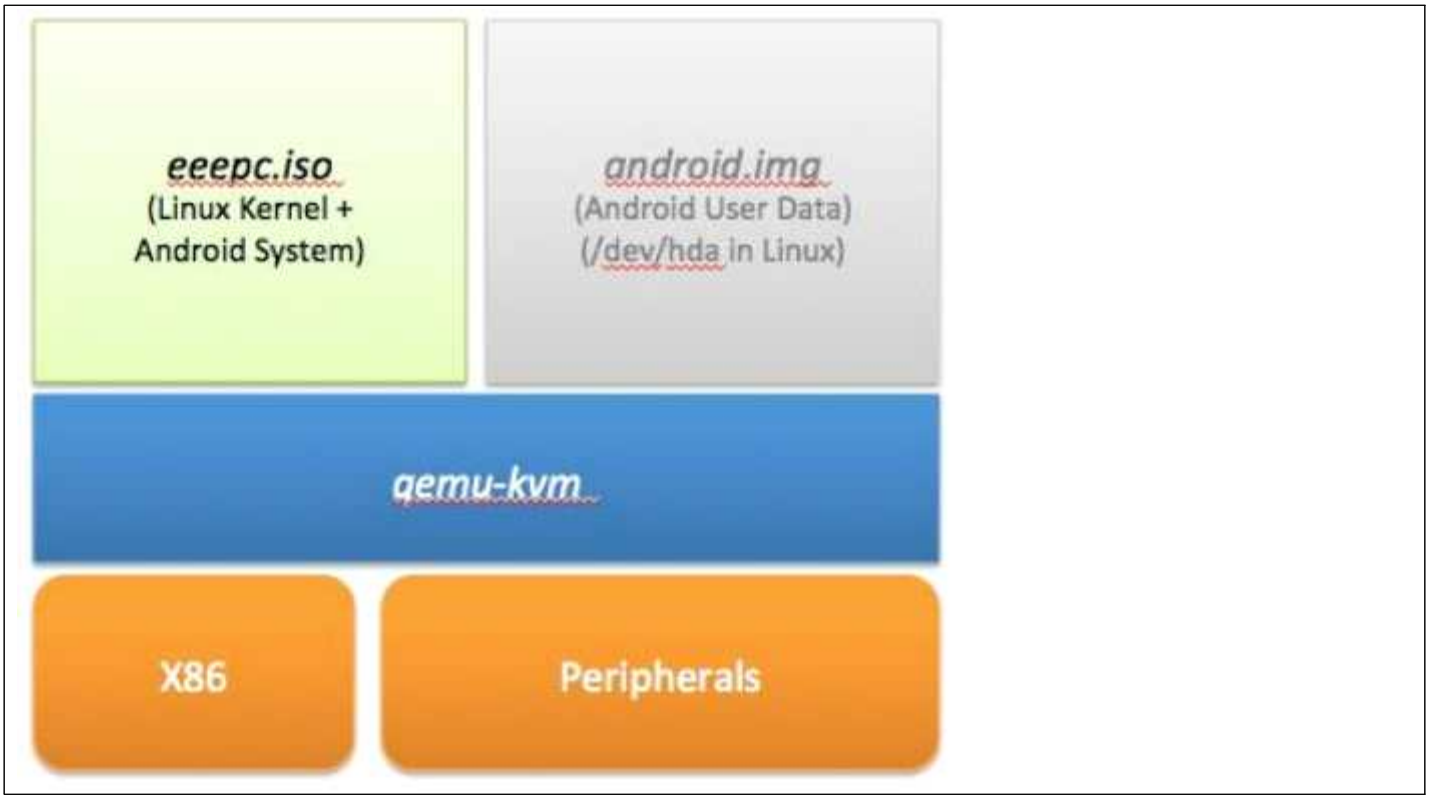
반가상화(Paravirtualization)는 전체 가상화와 약간 유사한 대중적인 기술이다. 이 방식은 기반 하드웨어로의 공유 액세스에 하이퍼바이저를 사용하지만, 가상화 인식 코드를 OS 로 통합한다.이 방식은 재장컴파일이나 트래핑(trapping)을 할 필요가 없다. OS 그 자체로 가상화 프로세스에 협력하기 때문이다. 앞서 언급했던 것처럼, Paravirtualization은 게스트 OS들이 하이퍼바이저에 맞게 수정되어야 한다. 이것이 단점이다. 하지만, Paravirtualization은 가상화되지 않은 시스템 성능에 가까운 성능을 보인다. 전체 가상화와 마찬가지로 여러 다른 OS 들이 동시에 지원된다.

오픈소스에뮬레이터

QEMU 소개

이 장에서는 가상화 기반 기술인 QEMU의 개념, 구조, 동작 원리, 디스크 관리, 네트워크 관리방법에 대해 알아볼 것이다. 또한 설치 및 사용방법에 대해 알아보고, 마지막으로 QEMU 를 이용하여 호스트 상에서 게스트 OS 를 설치하는 과정을 알아본다.

가장 각광받는 Xen이나 KVM보다 QEMU를 먼저 다루는 이유는 QEMU가 이 두가지 하이퍼바이저보다 먼저 개발되었고 이 두 하이퍼바이저는 내부적으로 QEMU를 디바이스 에뮬레이션 목적으로 활용하거나 QEMU의 성능을 가속시키는 방식으로 동작하기 때문이다. 그러므로 QEMU의 기본적인 동작 원리와 특정, 옵션을 제대로 파악해두면 다른 하이퍼바이저의 구조 및 동작 원리에 대한 본질적인 이해에 많은 도움이 될 것이다.

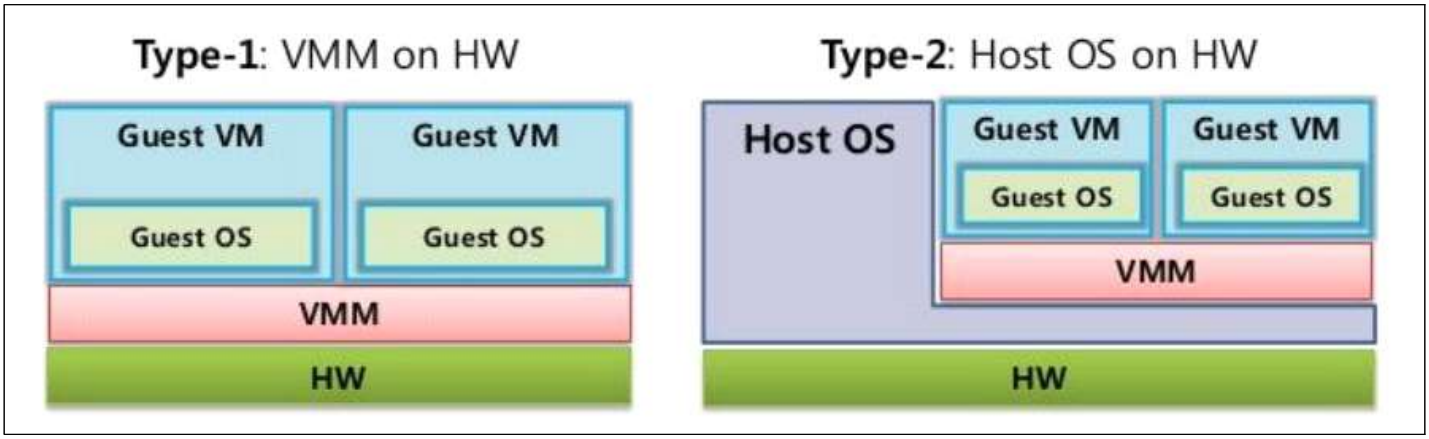


QEMU는고속의 동적 바이너리 변환 기법 [Dynamic Binary Translation]을 사용하는 프로세서 에뮬레이터이자 가상화 하이퍼바이저로서 Fabrice Bellard에 의해 개발되었다. 원래의 QEMU 는 x86 기반이 아닌 하드웨어에서 x86 기반의 리눅스 용용 프로그램들을 실행하는 목적으로 개발되었으나 지금은 개방형 아키텍처 구조로 x86, PowerPC, SPARC, ARM 계열 등의 다양한 프로세서를 지원한다.

바이너리 변환이란 하나의 명령어 셋을 다른 명령어 셋으로 코드 번역해주는 처리를 말하며 정적인 방법과 동적인 방법이 있다. 정적 바이너리 변환은 모든 실행 파일의 코드를 대상 아키텍처에서 실행되는 코드로 한 번에 변환하지만, 동적 바이너리 변환은 기본 실행 블록의 단위만큼만 코드를 순차적으로 읽어서 대상 아키텍처에서 실행되는 코드로 변환한다. QEMU가 에뮬레이터로 사용될 때에는 특정 하드웨어 기반에서 실행 가능하도록 작성된 OS 나 프로그램을 아키텍처의 변경 없이 다른 하드웨어에서 실행되도록 중계하는 역할을 한다. 게스트와 호스트의 CPU 아키텍처가 동일한 경우 원래는 명령어 변환이 필요없지만, 에뮬레이터는 무조건 한 번은 변환이 수행되므로 상당한 성능 저하가 일어난다. 하지만 게스트와 호스트의 CPU 아키텍처가 서로 다르고, 에뮬레이션하고자 하는 CPU의 원래 속도가 호스트 CPU 보다 느릴 때는 에뮬레이션으로 인한 속도 향상 효과를 얻을 수도 있다. QEMU가 가상화 하이퍼바이저로 사용될 때에는 KQEMU (또는 QVM86)를 통해 게스트 OS의 코드를 호스트 시스템의 CPU에서 직접 실행시키기 때문에 높은 성능이 발휘된다. 즉 커널코드는 동적 바이너리 변환을 통해 실행해주고. 사용자 코드는 네이티브 모드에서 실행해주는 가속 모드를 지원함으로써 빠른 속도를 보여준다. 가상화에서 게스트와 호스트의 CPU 아키텍처가 동일하거나 호스트 CPU에서 하드웨어적으로 직접 수행 모드를 지원하는 경우에는 게스트 CPU에서 수행되는 대부분의 명령어를 호스트 CPU에서 직접 수행할 수 있다. 또한 QEMU는 Xen 또는 KVM 하이퍼바이저의 가상화 기능을 도와주는 역할을 수행하기도 하는데, 이 경우 하드웨어 디바이스들의 에뮬레이터 기능을 수행하기 위할 변형된 형태의 QEMU 버전이 사용된다. 현재 Xen, KVM, 버추얼박스 등의 하이퍼바이저는 QEMU

의 소스 코드를 사용하거나 QEMIJ의 기능을 이용한다. 시리얼은 QEMU의 에뮬레이션 기능을 이용하여 시스템을 디버깅한다. QEMU 는 2023 년 8월 기준으로 8.0.4버전이 발표되었으며, 리눅스, 윈도우, 맥 OS X, FreeBSD, 오픈 솔라리스 같은 다양한 OS 와 ARM, MIPS 같은 다양한 칩셋을 지원하고 있다.
QEMU 는 바이너리 설치 후에 기본적으로 명령행에서 실행할 수 있다. 그래픽 사용자 인터페이스 환경을 제공하는 프로그램을 설치하면 GUI 상에서도 실행할 수 있다. 그래픽 사용자 인터페이스를 제공하는 버전은 리눅스용으로 AQEMU, qemu-launcher 등이 있으며 윈도우용으로는 'QEMU 윈도우 매니저', QGui 둥이 있고, 맥 OS X 용으로는 Q 가 있다. 이중 QEMU 윈도우 매니저는 QEMU 및 KQEMU 성능 가속화 패치를 이미 포함하고 었기 때문에 설치가 쉽다.

QEMU 구조의 이해
하드웨어 위에서 바로 실행되는 하이퍼바이저를 네이티브 또는 베어메탈 방식이라고 하며, 호스트 OS 위에서 실행되는 하이퍼바이저를 호스티드 방식이라고 한다. QEMU는 후자인 호스티드 방식에 속한다. 참고로 이와 같은 분류 기준으로 본 때 시트릭스의 XenServer, VMware의 ESXi, MS의 하이퍼- V 등이 전자에, QEMU, 버추얼박스, KVM 은 후자에 속한다.
아래의 그림을 살펴보기 바란다.



QEMU 모드
QEMU 는 전체 시스템 모드는 물론 사용자 모드 에뮬레이션을 지원한다. 각각의 특성은 다음과 같다.
전체 시스템 에뮬레이션 모드
CPU 및 주번 장치들을 포함하는 전체 시스템 하드웨어를 에뮬레이션한다. 즉, QEMU는다양한 하드웨어 플랫폼 및 게스트 OS 를 실행할 수 있으며, 동적 변환을 사용하기 때문에 속도도 빠르다. 원래 이 모드는 여러 가상 CPU를 실행하기 위해 단일 스레드만을 지원하지만 COREMU라는 패치 덕분에 이러한 제한이 없어졌다. 이 패치가 적용된 QEMU는 각 CPU에 대한 바이너리 변환 엔진을 독립적인 인스턴스로 실행하고, 작은 라이브러리 계층이 CPU 및 디바이스 간의 통신을 담당하여 멀티 스레드를 처리한다.

사용자 모드 에뮬레이션
사용자 모드 에뮬레이션에서 QEMU는 특정 아키텍처의 CPU 상에서 컴파일된 프로세스를 다른 CPU 상에서 실행하도록 해준다. 빠른 크로스 컴파일 또는 크로스 디버깅이 주요 사용 목적이며 기본적으로 다중 스레드를 지원하기 때문에 동시에 여러 가상 CPU 를 실행할 수 있다. 예를 들어 이 모드는 와인 윈도우에뮬레이터 실행에 사용할 수 있다.

CPU의 동작 모드는 개념적으로 커널 모드와 사용자 모드로 구분할 수 있는데 이는 권한이 가장 많은 Ring 0 부터 가장 적은 Ring 3 까지의 여러 단계로 구분된다. 먼저 커널 모드는 Ring 0에 할당하며 이 모드에서는 CPU의 모든 명령어 셋, I/O 및 메모리 제어 등을 실행할 수 있다. OS의 커널은 보통 모든 접근 제어권한이 필요하기 때문에 Ring 0에서 동작한다. 반면에 사용자 모드는 Ring 1~3에 할당하며 이 모드에는커널 모드에서 실행 가능한 명령어 셋의 일부만을 실행할 수 있다. 사용자 응용 프로그램들은 보통 Ring 3에서 실행된다. 최근의 인텔 및 AMD CPU 는 각각 인텔-V 와 AMD-V 기술을 통해 Ring 1에서 하이퍼바이저가 직접 Ring 0의 하드웨어 접근을 제어할 수 있는 x86 가상화 명령어 셋을 제공한다. 따라서 이 기술을 적용하면 게스트는 Ring 1에서 실행되어서 호스트 OS의 도움 없이도 Ring 0에서 실행되는 하이퍼바이저를 통할 직접 Ring 0의 명령어들을 실행할 수 있다

QEMU 내부 기능

QEMU 는 아래의 그린과 같이 다양한 기능을 수행하는 요소로 구성되어 있다.

QEMU 내부 기능도

CPU 에뮬레이터
X86, PowerPC, SPARC, ARM 등의 다양한 CPU 를 가상으로에뮬레이션하는 기능을 수행한다.

디바이스 에뮬레어터
IDE 하드디스크, 키보드, 릴우스, VGA 디스플레이, NE2000 네트워크 카드 등의 다양한 장치를 에뮬레이션한다.

일반 디바이스 처리기
블록 디바이스, 네트워크 디바이스 등의 일반 장치들을 호스트 장치들에 연결하는 기능을 수행한다.

머신 기술자
PC, PowerMac, Sun4m과 같이 에뮬레이션된 장치들을 설명하는 기능을 수행한다.

디버거
QEMU를 디버깅 용도로 사용할 수 있도록 해준다.

사용자 인터페이스
QEMU의 명령행 인터페이스를 구성한다.

KQEMU

QEMU 는 QEMU의 성능을 높여주는 리눅스 커널 모듈 패치로서 QEMU와 마찬가지로 Fabrice Bellard가 개발했다. KQEMU 는 게스트와 호스트의 OS 아키텍처가 동일한 경우 x86 기반의 게스트 OS에 대해서 하드웨어의 지원을 받아 유저 모드와 일부 커널 모드 코드를 호스트 머신의 CPU에서 직접 실행하고, 프로세서 및 주번 장치에뮬레이션은 커널 및 리얼 모드 코드에 대해서만 적용하여 게스트 OS의 성능을 높여준다.

QEMU는 x86 또는 x86j54 기반 리눅스 2.4 나 2.6 호스트에서 지원되며, FreeBSD 와 윈도우 NT/2000/2003/XP에서는 실험용 버적을 지원한다. KQEMU가 비공개 소스코드였을때. 이에 대한 오픈소스 대안으로 QVM86 이 발표되었으나 KQEMU 가 1.3.0 pre10 버적부터 GPL로 공개된 이후 개발이 중단되었다. 현재 QEMU vO.II 이상의 버전에서는 KQEMU를 더 이상 지원하지 않고 KVM 을 통한 가속 기능만을 지원하고 있다.

QEMU 와 하이퍼바이저

QEMU 와 관련된 하이퍼바이저로 Bochs, KVM, Xen, 버추얼박스가 있다.

Bochs 와 QEMU

맨드리바사의 LGPL 오픈소스 소프트웨어인 Bochs 는 DOS, 윈도우, BSD, 리눅스 등의 x86/x86_64 기반의 게스트 OS 를 지원하는 에뮬레이터로, 가상화를 지원하지 않아 속도가 매우 느리다. 하지만 Bochs 는 순수한 에뮬레이터이기 때문에 호스트 시스템이 32비트인지 64 비트인지에 상관없이 다양한 게스트 OS를 실행할 수 있고 마우스, 그래픽 카드, 키보드, 네트워크 카드 등의 주번 장치들을 에뮬레이션 해주고 쉬운 디버깅 환경을 제공하기 때문에 주로 OS 분석 및 테스트 용도로 사용된다.

반면에 QEMU는 GPL/LGPL 오픈소스 소프트웨어고, x86/x86-64, ARM, SPARC,PowerPC 등의 다양한 게스트 OS 를 지원하는 에뮬레이터이자 가상화 하이퍼바이저이기 때문에Bochs 보다 속도가 훨씬 빠르다. Bochs 와 QEMU의 최신 버적은 각각 2.5.1 과 1.0.1 로 최근까지도 안정화된 버전이 출시되고 있다.

KVM 과 QEMU

KVM 은 리눅스 커널 모듈에 포함된 드라이버 형태의 하이퍼바이저다. KVM 하이퍼바이저에서의 QEMU는 KVM에 최적화되어 변형된 버적인 qemu-kvm이 실행되어서 x86 시스템에 대한 에뮬레이터의 역할을 수행한다. QEMU는 호스트와 동일한 아키텍처의 OS를 실행하고자 할 경우에만 (예를 들어 호스트 및 게스트 OS 가 동일한 x86 시스템인 경우) KVM의 가속 기능을 이용한다.

KVM도 KQEMU 처럼 QEMU의 성능을 가속해주는 리눅스 커널 모듈이지만 KQEMU는 호스트 시스템이 반드시 CPU 가상화를 지원할 필요는 없다는 점이 다르다. 하지만 성능면에서는 CPU 가상화를 지원하는 경우에만 동작하는 KVM이 더 빠르다. KVM 은 완전한 가상화를 지원하기 위해 하드웨어 가상화 기능을 이용하기 때문에 X86 기반에 리눅스 커널 2.6.22 버전 이상을 사용하고 인텔-VT 또는 AMD-V 가상화를 지원하는 CPU가 장착된 시스템에서만 사용할 수 있다.

Xen 과 QEMU

Xen 은 변형된 게스트 OS 를 실행하는 반가상화와 인텔- VT 또는 AMD -V CPU 기반의 완전 가상화를 지원하는 하이퍼바이저다. Xen 하이퍼바이저에서는 변형된 QEMU인 qemu-dm이 실행되는데, QEMU 는 Xen-HVM에서 디바이스에뮬레이션과 일반 디바이스 간의 통신을 담당하여 가상 I/O 요청을 처리한다. KVM 과는 달리 Xen에서는 가상의 CPU를 QEMU가 아닌 하이퍼바이저가 직접 관리한다.

버추얼 박스와 QEMU

버추얼박스는 2007 년 처음 발표되었으며, 현재는 썬 마이크로 시스템즈를 인수한 오라클에 의해 관리되는 GPL 기반의 공개 소프트웨어다. 버추얼박스는 QEMU의 가상 하드웨어 디바이스룰 사용하며 자체의 QEMU 기반 동적 재 컴파일러가 포함된다. 하이퍼바이저는 게스트의 거의 모든 코드들을 호스트 OS에서 실행하며 일부 특수한 경우에만 재컴파일러를 사용한다. 하이퍼바이저의 내부에서 역어셈블러가 동작하여 코드 분석 및 패치가 실행되므로 재컴파일러는 최소한으로 사용된다.

QEMU 실행옵션
QEMU의 명령행 및 그래픽 모드에서 실행할 때 사용하는 옵션에 대해서 정리해 보도록 하자.

명령행 옵션
명령행에서 QEMU를 사용하는 방법은 다음과 같이 간단하다.
크게 옵션과 '디스크_ 이미지'로 구성되는데 옵션이 상당히 많다.

```
사용법: qemu [옵션들] [디스크_이미지]
```

디스크_이미지
IDE 하드디스크 0의 raw 하드디스크 이미지다.
어떤 타깃들은 디스크 이미지가 필요 없는 경우도 있다.

표준 옵션
-h
도움말 화면출력
-version
버전 정보 출력
-machine [type=] name[,prop = value []]
에뮬레이션할 머신을 이름으로 선택한다. ('-machine ?'를 입력하면 가능한 머신 종류를 출력한다).
-CPU model
CPU 모델을 선택한다. (-cpu를 입력하면 CPU 리스트 및 추가적인 기능 선택 사항을 출력한다). -smp n[,cores=코어 개수] [,threads=스레드 개수 [,sockets=소켓 개수] [,maxcpus=최대 CPU 개수] CPU 가 n개인 SMP 시스템을 시뮬레이션하며 타깃 PC에서는 최대 255 개의 CPU가 지원된다. Sparc32 타깃에서 리눅스 OS 는 최대 4 개의 CPU를 지원한다. PC 타깃에서는 소켓당 코어 개수와 코어당 스레드 개수악 총 소켓 개수를 지정할 수 있고 빠진 값은 자동으로 계산되며, 세 개의 값들 중 한 값만 지정하면 총 CPU 개수를 생략할 수 있다. maxcpus 는 핫플러그 가능한 최대 CPU 개수를 말한다.
-numa opts
멀티 노드 NUMA 시스템을 시뮬레이션하며. mem 과 cpus 값을 지정하지 않으면 리소스는 균등분배된다.
-fda 파일, -fdb 파일
파일을 플로피디스크 0/1 이미지로 사용한다. 'dev/fdO'을 파일에 입력하면 호스트 머신의 플로피 디스크를 사용할 수도 있다. -hda 파일, -hdb 파일, -hdc 파일, -hdd 파일 파일은 하드디스크 0, 1, 2 또는 3 이미지를 지정할 수 있다.

QEMU 디스크 관리

QEMU 가 지원하는 파일 포맷에 대해 알아보고 디스크 관리 방법에 대해 살펴본다. QEMU 는 0.6.1 버전 이후부터 섹터에 값이 씌어질 때마다 디스크의 크기가 증가하는 가변 이미지 형식, 압축 형식, 암호화된 형식 등 많은 디스크 이미지 포맷들을 지원한다. QEMU 0.8.3 버전에는 VM의 스냅샷을 지원하는 데에 필수적인 qcow2 디스크 이미지 포맷이 추가되었다. 디스크 이미지 포맷에 대한 자세한 내용은 qemu-img 명령행의 명령어 옵션을 설명하면서 다루겠다.

qemu-img

qemu-img 는 QEMU의 디스크 이미지 관리를 위한 명령어다. 사용 방법은 다음과 같다.

```
qemu-img 명령어 [명령어 옵션들]
```

가능한 명령어 및 명령어 옵션들은 다음과 같다.

```
check [-f 포맷] 파일명
create [-f 포맷] [-0 옵션들] 파일명 [크기]
commit [-f 포맷] 파일명
convert [-c] [-f 포맷] [-0 출력 포맷] [-0 옵션들] 파일명 [파일명 2 [...]]출력 파일명
info [-f 포맷] 파일명  snapshot [-1 | -a 스냅샷 | -c 스냅샷 | -d 스냅샷] 파일명
```

파일명

디스크 이미지 파일명

포맷

디스크 이미지의 형식을 말하며 대부분의 경우 자동적으로 파악된다. 지원되는 디스크 형식들에 대해서는 아래를 참고한다

크기

디스크 이미지의 바이트로 단위 크기다. K 또는 k(킬로바이트 1.024). M(메가바이트 1.024k). G(기가바이트 1.024M). T (테라바이트 1,024G)를 추가로 불일 수 었으며, b 옵션은 무시된다.

출력 파일명

최종 디스크 이미지의 파일명

출력 포맷

최종 출력 형식

옵션들

콤마로 구분한 포맷 관련된 옵션들의 목록이며, [이름=값]의 형식으로 표시한다. - 0 ? 옵션을 사용하면 사용되는 포맷이 지원하는 옵션을 보여준다. 예를 들어 아래의 예제와 같은 방식으로 할당 포맷에 대한 구체적인 옵션들을 확인할 수 있다.

```
# qemu-img create -f qcow2 -o ? test.qcow2 200k
```

- c

출력 이미지를 압축하도록 지정한다(qcow 포맷에만 할당).

-h

단독으로 또는 명령어와 함께 사용하여 도움말을 출력하고 지원하는 포맷들을 나열한다.

명령어 설명

```
create [-f 포맷] [-o 옵션들] 파일명 [크기]
```

크기와 포맷을 지정하여 새로운 디스크 이미지 파일명을 생성한다. 파일 포맷에 따라 추가적인 기능들을 활성화하는 한 개 이상의 옵션을 덧붙일 수도 있다. backing_file 옵션을 추가하면 이미지에는 backing_file과의 차이점만 기록하며, 이 경우 크기는 지정할 필요가 없다. commit 모니터링 명령어 또는 qemu-img commit 명령어를 사용하지 않는 한 backing_file 은 절대로 변경되지 않는다. 파일의 크기도 -0 크기 옵션을 사용하여 지정할 수 있다. 하지만 이 경우에는 꼭 지정하지 않아도 된다.

commit [-f 포맷] 파일명

파일명에 기록된 변경 사항을 베이스 이미지에 저장하여 반영한다.

convert [-c] [-f 포옛] [-0 출력 포맷] [-0 옵션들] 파일명 [파일명 2 [...]] 출력 파일명

출력 포맷 형식을 사용하여 파일명 디스크 이미지를 출력 파일명으로 변경한다. 선택적으로 -c 옵션을 주어서 압축하거나 암호화의 경우 -o 옵션을 주는 것처럼 특정 포맷과 연관된 옵션들을 지정할수도 있다. qcow 와 qcow2 포맷의 경우에만 압축을 지원하며 압축된 데이터는 읽기만 가능하다. 즉, 압축된 섹터가 재기록되는 경우 압축되지 않은 데이터로 저장된다. 이미지 변환은 qcow 또는 cow 와 같이 증가 가능한 포맷을 사용하여 빈 섹터들을 감지하고 출력 이미지에서 제거하는 방식으로 좀 더 작은 크기의 파일을 생성할 때에 유용하다. backing_file 옵션을 사용하면 지정된 베이스 이미지에 대해 쓰면서 복사하기(copy on write) 방식으로 출력 이미지를 생성한다. backing_file 은 입력의 베이스 이미지와 같은 내용이어야 하지만 파일 경로, 이미지 포맷 등은 다를 수도 있다.

```
info [-f 포맷] 파일명
```

파일명 디스크 이미지에 대한 정보를 출력한다. 파일이 차지하는 크기와 함께 실제 디스크가 차지하는 크기를 알 수 있다. 만일 VM의 스냅샷들이 디스크 이미지에 저장되어 있다면 이들 정보도 함께 표시된다.

```
snapshot [-l | -a 스냅샷 | -c 스냅샷 | -d 스냅샷] 파일명
```

스냅샷

파일명 이미지에 대할 생성, 적용, 삭제할 스냅샷의 이름

-a

파일명 이미지에 대할 스냅샷을 적용한다(디스크를 저장된 상태로 되돌린).

-c

파일명 이미지에 대한 스냅샷을 생성한다.

-d

파일명 이미지에 대한 스냅샷을 삭제한다.

-l

파일명 이미지에 대한 모든 스냅샷을 나열한다.

이 옵션들을 사용하면 모든 디스크 이미지는 읽기만 가능한 상태로 간주되며 섹터들은 임시 파일로 /tmp에 기록된다. raw 디스크 이미지에 다시 강제로 저장하도록 하기 위해서는 commit 모니터 명령어를 사용하면 된다.

이미지 포맷들

raw

raw 디스크 이미지 포맷(기본값). 이 포맷은 구조가 간단하여 다른 에뮬레이터에도 쉽게 적용된다. 대상 파일시스템이 홀(hole)을 지원한다면(예를 들어 리눅스의 ext2 또는 ext3, 윈도우의 NTFS) 기록된 섹터들만 공간을 차지할 것이다. 실제로 기록된 크기를 알기 위할서는 qemu-img info 또는 유닉스/리눅스의 ls -ls 명령을 사용하면 된다.

host_device

호스트 디바이스 포맷. 이 포맷은 홀(hole)이 지원되지 않는 블록 디바이스 또는 다른 디바이스로 변경하려고 할 때 raw 포맷 대신 사용되어야만 한다.

qcow2

현재 사용되는 QEMU의 이미지 형식으로서 가장 많은 기능을 가지는 포맷이다. 윈도우와 같이 대상 파일시스템이 홀을 지원하지 않는 경우 더 작은 크기의 이미지를 생성하거나, 선택적인 AES 암호화, zlib 기반 압축, 다중 VM 스냅샷 생성 등의 목적으로 사용할 수 있다.

지원되는 옵션은 다음과 같다.

backing_file

베이스 이미지의 파일명 ('create' 하위 명령 참조)

backing_fmt

베이스 이미지의 파일 포맷

encryption

이미지 암호화 설정. 암호화는 128 비트 키를 사용하여 매우 안전한 AES 포맷을 사용한다. 최고의 안전성을 위해서는 16자의 긴 패스워드를 사용하면 된다.

cluster_size

qcow2 클러스터 크기를 변경한다(512 와 2M 사이값). 클러스터의 크기를 작게 할수록 이미지 파일의 크기를 줄일 수 있지만 일반적으로 이 값을 크게 할 수록 더 좋은 성능이 발휘된다.

preallocation
선 할당 모드(off 나 metadata 만 설정 가능). 선 할당된 메타데이터는 초기 크기가 크지만 이미지의 크기를 증가시켜 손쉽게 성능을 향상시킬 수 있다.

qcow
이전 QEMU 이미지 포맷과의 호환성을 위해 존재한다.
지원되는 옵션은 다음과 같다.

backing_file
베이스 이미지 파일명 ('create' 하위 명령어 참조)

encryption
이미지를 암호화하도록 설정한다.

qemu-nbd
NBD 란 외부 호스트의 블록 디바이스에 연결하여 사용하는 장치를 말한다. QEMU 하이퍼바이저는 qemu-img 를 이용하여 생성된 QCOW 또는 QCOW2 디스크 이미지를 자체적으로 제공하는 qemu-nbd 서비스를 통해 NBD 서비스를 가상 머신에 제공한다. 이제부터 QEMU에서 NBD 를 어떻게 사용하는지 알아보자.

명령어 설명
이 명령어는 NBD 네트워크 블록 디바이스 서버를 실행하여 QEMU의 디스크 이미지를 외부에서 접근할 수 있게 해준다.

사용법: qemu-nbd [옵션]... 파일명

파일명 디스크 이미지 파일명
- P , - port=PORT 대기할포트번호(기본값 10809)
- o, - offset=OFFSET 이미지에 대한 오프셋 설정값
- b, bind=IFACE 바인드를 수행할 인터페이스 지정(기본값 aO.O.O)
- k, - socket=PATH 유닉스소켓의 경로(기본값/var/lock/qemu-nbd -DEVICE)
- r, read -only 읽기 적용 모드로 외부에 노출한다.
- P, partition =NUM 파티션 NUM만 외부에 노출한다.
- s, snapshot 스냅샷 파일을 사용한다.
- n, nocache 호스트의 캐시를 비활성화한다.

사용 예제
NBD 를 사용하여 QEMU를 이용하는 방법을 간단한 예를 들어 설명하면 다음과 같다. 먼저 다음과 같이 디스크 이미지 파일인 test_nbd.qcow2 를 준비해놓고 qemu-nbd 를 실행한다.

```
# qemu-nbd ./test_nbd.qcow2
```

이때 사용할 소켓을 아래 예제의 /tmp/test_nbd_socket 과 같이 직접 지정할 수도 었다.

```
# qemu-nbd --socket=/tmp/test_nbd_socket 7test.qcow2
```

이때 다음과 같이 여러 게스트 사이에 NBD 를 공유하기 위해서 --share 옵션을 지정할 수 었다

```
# qemu-nbd --socket=/tmp/test_nbd_socket --share=2 ,/test.qcow2
```

QEMU 네트워크 관리

QEMU 는 다양한 네트워크 카드(PCI 또는 ISA)를 에뮬레이션하고 이를 여러개의 가상 네트워크에 연결할 수 있다. 호스트 머신의 TAP 장치들은 임의의 QEMU VLAN에 연결될 수 있다. 서로 다른 QEMU 인스턴스들을 VLAN 으로 연결하면 대규모 네트워크를 시뮬레이션할 수도 있다. 예를 들어 non-privileged 사용자 네트워킹 모드는 기본 네트워크 연결을 위한 TAP 장치를 대체할 수 있다.

가상 네트워크 디바이스

가상 네트워크 디바이스는 에뮬레이션할 하드웨어의 구성에 따라 게스트 OS의 환경에 맞게 선택해야 한다. 예를 들어 PCI 버스를 가지는 하드웨어는 선택의 폭이 다양하다. QEMU의 기본 네트워크 어댑터는 el000 이며, QEMU-KVM에서는 rtl8139 인데 게스트 OS에 맞게끔 최적화된 반가상화 드라이버인 virtio-net 을 사용할야 가장 좋은 성능을 낸다. 게스트 OS에 특정 가상 네트워크 디바이스를 추가하기 위해서는 -device 옵션을 사용한다. 1.0 이전 버적에는 이 목적을 위한 -device 옵션과 현재는 쓰지 않지만 호환성을 위해서 지원하는 -net nic 옵션이 있다. 물론 QEMU에는 다른 디바이스를 선택하기 위한 옵션도 있다. 예를 들어 새로운 디바이스를 추가하는 명령은 다음과 같다

```
-device 디바이스명,netdev=네트워크 ID,macaddr=맥 주소, 디바이스 옵션들
```

디바이스명은 디바이스를 지정하기 위할 사용하고 (예를 들어 인텔의 i82559C 이더넷 장치를 사용하기 위해서는 i82559c 입력), 네트워크 ID 는 디바이스를 위한 네트워크 식별자를 지정하기 위할 사용하고, 맥 주소는 디바이스의 맥 주소를 지정하기 위할 사용하며, 디바이스 옵션들은 특정 디바이스를 사용하기 위한 옵션들을 지정하기 위해서 사용한다.

오픈소스 하이퍼바이저 KVM 소개

이 장에서는 가상화 솔루션인 KVM에 대해서 다룬다. KVM은 전가상화 방식을 사용하기 때문에 윈도우 XP 와 Centos 모두를 게스트로 설치하고 관리할 수 있다.

게스트 설치 준비

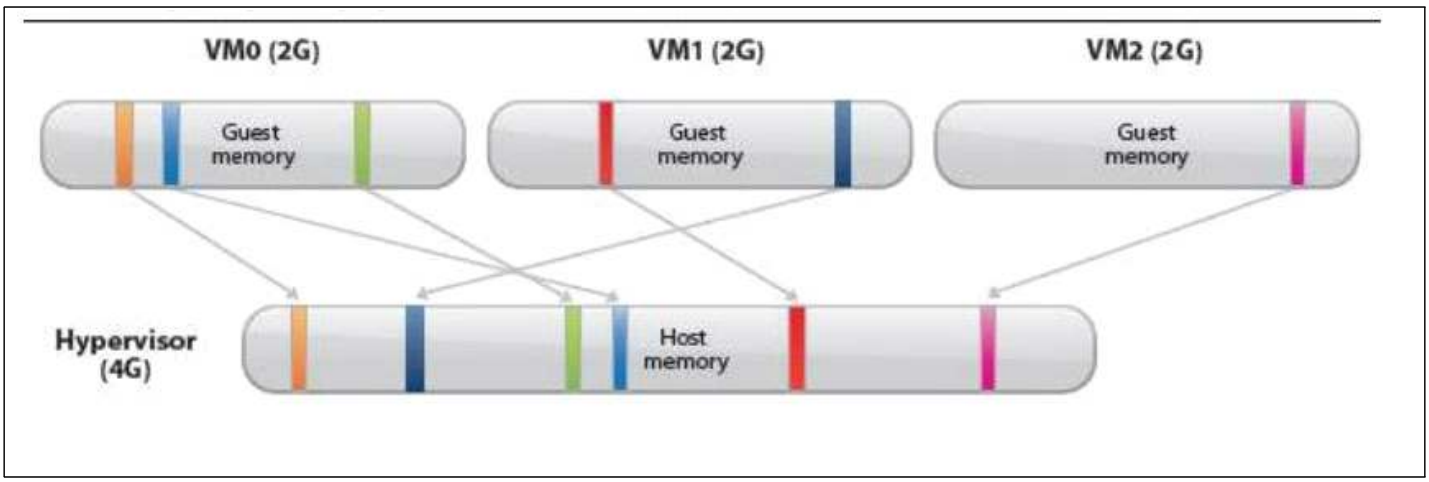
어느 OS 나 마찬가지지만 윈도우나 리눅스 할 값 없이 OS를 설치하고 나서도 어느 정도 여유공간이 있어야 한다. 따라서 디스크는 최소 6GB, RAM 메모리는 최소 2GB 를 확보해야 제대로 성능을 낼 수 있다. 그런데 OS 를 제외하고 6GB의 여유 공간이 있는 편이 좋다. 또한 가상화 서버마다 가상 CPU 및 하이퍼바이저용 프로세서 코어와 하이퍼스레드도 필요하다.

권장 사항은 다음과 같다.

저장소 6GB + 게스트용 공간

CPU 가상화 서버당하나이상

메모리 2GB 이상(오버커밋 Overcommit) KVM 은 오버커밋을 지원한다. 오버커밋이란 실제 제공되는 한정된 물리적인 메모러 크기보다 가상화된 게스트들에 할당된 메모리 크기의 합이 더 클 수 있게 지원하는 기능이다. 가상화 CPU에도 이를 적용할 수 있으며, KVM 이 안정적으로 운영해준다. 아래의 그림을 살펴보면 물리적인 메모리 크기가 2GB인데 게스트에 할당된 메모리는 총 3GB 다. 오버커밋이 지원되기 때문에 가능한 일이다.



KVM 지원 지원 여부 확인

윈도우 계열은 윈도우 95 부터 거의 모든 종류를 지원한다.

KVM은 전가상화의 지원을 받기 때문에, 하드웨어에서 몇몇 조건을 제공해야 한다. 하드웨어라고 하면 물리적으로 복잡하게 느껴지지만, 하드웨어란 실상 CPU 를 말한다

```
[root@vmtest]# egrep '(vmx|svm)' --color=always /proc/cpuinfo
```

```
upubuntu@ubuntu:~$  egrep --color "(vmx|svm)" /proc/cpuinfo
flags           : fpu vme de pse tsc msr pae mce cx8 apic mtrr pge mca cmov pat
pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx lm constant_tsc arch_pe
rfmon pebs bts aperfmperf pni dtes64 monitor ds_cpl vmx est tm2 ssse3 cx16 xtpr
pdcm lahf_lm dts tpr_shadow
flags           : fpu vme de pse tsc msr pae mce cx8 apic mtrr pge mca cmov pat
pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx lm constant_tsc arch_pe
rfmon pebs bts aperfmperf pni dtes64 monitor ds_cpl vmx est tm2 ssse3 cx16 xtpr
pdcm lahf_lm dts tpr_shadow
upubuntu@ubuntu:~$
```

단적으로 이야기하면 인텔 칩셋은 VMX, AMD 칩셋은 SVM 을 지원해야 KVM 을 설치하여 가상화 기능을 쓸 수 있다. 만약 CPU의 사양에서 가상화를 지원하는데 위의 그림처럼 egrep '(vm xlsvm)' — color=always /proc/cpuinfo 명령을 실행했을 때 flags 정보에 VMX 나 SVM 이 표시되지 않으면 BIOS 설정에서 가상화 확장 기능이 비활성화로 설정되어 있는 지 확인하기 바란다.

BIOS 가상화 확장 기능의 설정은 주로 프로세서를 설정하는 Chipset,Advanced CPU Configuration or Northbridge 와 같은 항목에서 찾아볼 수 있다. 설정을 완료했으면 재부팅하고 나서, 명령어를 다시 실행하여 VMX/SVM 이 출력되는지 확인하자. 그럼에도 문구가 출력되지 않으면 가상화 기능이 지원되지 않거나 BIOS 설정이 올바르게 되지 않은 것이다. 자세한 내용은 CPU의 설명서에서 확인하기 바란다.

지원 저장장치

KVM 은 다양한 저장 장치를 지원한다.

서버의 로컬에 있는 저장소, 물리적인 디스크, 로컬로 연결된 물리적 LUN, LVM (Logical Volume Manager), iSCSI, 파이버 찾널(Fiber Channel)기반의 LUN 등이 있다.

보안정책 설정

SELinux 는 리눅스의 보안 정책을 강화시키기 위한 방법 중 하나다. 보안이 강화되지만 사용성을 해친다. SELinux 가 활성화된 상태에서 정해진 디렉터리에 이미지가 존재하지 않으면 SELinux 는 게스트 이미지를 불러오지 못한다. SELinux에서는 모든 게스트 이미지가 /var/lib/libvirt/images에 저장되어 있어야 한다. SELinux의 보안 정책을 변경하는 방법은 다음과 같다.

```
1 vi /etc/sysconfig/selinux 명령으로 selinux 설정 파일 열기
2 enforcing 을 permissive 로 변경 (SELINUX=enforcing => SELINUX=permissive)
3 저장후 장부팅
```

KVM 패키지 설치

KVM 은 리눅스 배포판의 커널에 포함되어 있기 때문에, 리눅스를 설치할 때 가상화 패키지를 선택하여 설치할 수 있다. 혹은 리눅스 배포판을 설치할때 KVM 기능을 설치하지 못했더라도 너무 걱정 말자. 운영 중인 리눅스에 추가로 설치할 수도 있다.

OCI는 무엇인가?
OCI는 Open Container initiative의 약자이며, 오픈소스 기반의 컨테이너들의 표준 사항을 정의한다. 리눅스 파운데이션은 오랜기간 컨테이너 방향에 고민이 많았으며, 결정적으로 OCI가 탄생하게 된 이유는 Docker로 인하여 가속화가 되었다.

기존 Docker와 OCI 컨테이너의 차이점
Docker/OCI
Docker는 리눅스에서 처음으로 통합적인 컨테이너 기술을 제공하였다. 도커는 제작은 2010년도에 인큐베이터 회사로 시작 하였으며, 2011년도에 정식적으로 런칭을 하였다. 이때 프로젝트 이름은 dotCloud라는 이름으로 시작을 하였으며, 이 회사는 컨테이너 기반의 platform-as-a-service를 목표로 하고 있었다.

도커가 대중에 공개가 된 것은 2013년 파이콘(PyCon)에서 공개가 되었으며, 공식적으로 오픈소스 기반으로 릴리즈가 된 것은 2013년도에 오픈소스로 전환을 하였다. 이때 당시 리눅스는 LXC, chroot같은 도구로 컨테이너 환경을 구성하고 있었으며, 이 시점에 릴리즈가 된 도커 버전은 0.9버전으로 기존의 LXC를 대신하여 오픈소스 커뮤니티에서 받아 들이기 시작하였다. 도커가 완전히 오픈소스기반으로 공개 및 개발이 된 시점은 2017년도이며, 이때쯤 오픈소스는 OCI라는 표준 사양을 만들면서 컨테이너 기술의 표준사항을 정의하기 시작하였다.

도커는 하나의 제품에서 컨테이너 런타임, 컨테이너 이미지 관리, 컨테이너 관리 및 컨테이너 이미지 레지스트리 이미지 서버를 제공한다. 현재 도커에서 사용하는 도커 컨테이너 이미지는 OCI에서 표준 이미지로 지정이 되었으며, 이미지 제작시 사용하는 명령어(Instruction) 및 관리시 사용하는 명령어는 기존 도커의 명령어를 그대로 채용하여 사용하고 있다.

OCI는 기본적으로 도커와 같은 컨테이너 기능을 제공하나, 도커의 기능을 하나씩 분리하여 독립적인 소프트웨어 기반으로 구성이 되어있다. OCI에서 제공하는 도구는 기본적으로 아래와 같다.

CRI-O
Container Runtime Interface의 약자, 기본적인 컨테이너 환경을 제공하는게 주요 목적이다. 이 컨테이너 환경은 쿠버네티스 혹은 오픈시프트와 같은 클러스터에서 사용하는 런타임이다. CRI-O는 도커보다 매우 가볍다는 장점이 있으며, 기본적인 컨테이너외에는 다음 프로그램을 이용한다.

Podman
Buildah
Skopeo

cri-o/podman과 docker 아키텍처 및 기능
ROOTLESS/ROOTFUL 컨테이너

CRI 런타임 역할은 컨테이너 기본적인 컨테이너 생성 및 조회를 할 수 있도록 환경을 조성해주는 역할이다. 컨테이너를 생성하면 CRI는 OCI를 통해서 runc기반으로 컨테이너를 생성한다. 이러한 구성으로 CRI와 Podman의 기능 및 관리 범위에 차이가 있다.

기본적으로 두 개의 제품은 컨테이너 생성 및 관리가 목적이나, CRI는 빠르고 가볍게 단순한 계층으로 컨테이너를 생성하는게 주요 목적이며, 이를 통해서 취약점의 최소화 및 빠르게 컨테이너 생성에 대한 목표를 가지고 있다. 두 개의 컨테이너는 하이레벨 컨테이너 런타임에 포함이 되며, 각자 담당하는 계층 위치는 다르다. 위의 컨테이너 다이어그램 참고로 좀 더 설명을 하겠다.

로우레벨 구역은 일반적으로 컨테이너가 구성 및 환경을 제공해주는 소프트웨어 계층이다. 이 계층은 여러 단계로 지원한다. 먼저 순수 컨테이너 런타임을 구성하는 runc자원이다. runc는 리눅스 소프트웨어 기반으로 동작을 하며, 기본적으로 커널기능 기반으로 동작한다. 커널에서 제공하는 기능은 기본적으로 다음과 같다.

Namespace
네임스페이스 기능은 시스템에 동작하는 프로세서를 시스템과 분리하여 독립적으로 동작하도록 격리를 한다. 주요 목적은 프로세서의 격리가 주요 목적이다. 주요 격리하는 자원은 IPC, net, cgroup, mnt, pid, pid_for_children, user, uts같은 자원이다. 하지만, 이의 자원도 커널 버전에 따라 범위가 다르다. 위의 자원은 커널 4.18.0EL 기반으로 작성이 되어 있다.

C-Group은 리눅스 커널에서 프로세스 사용하는 자원에 대해서 제한한다. 자원을 제한하는 방법은 앞에서 잠깐 이야기 하였지만, 제한(Limit), 격리(isolate)을 제공한다. 이를 통해서 모든 자원들이 서로 자원 경쟁을 하지 않으며 각각 프로세스 영역에 대해서 격리 및 추적이 되므로 시스템에 좀 더 안정적인 자원 운영 기능을 제공한다. 현재 RHEL 7,8버전으로 넘어 오면서 사용하는 C-Group버전은 v2이다.

컨테이너 보안을 위해서 다음과 같은 기능을 제공한다. 리눅스 배포판별로 다르지만 강화된 보안 기능을 다음과 같이 제공한다.

SELinux는 미국 NSA 및 레드햇에서 제작하여 리눅스 커뮤니티에 제공하였으며, 레드햇은 이를 맨처음에 도입하여 현재는 모든 레드햇 리눅스 계열의 배포판에 커널수준에서 보안 기능을 제공한다.
SELinux리눅스의 퍼미션과 비슷한 동작 방법을 가지고 있다. 다만, 동작조건은 리눅스 표준 퍼미션과 다르게 레이블링(Labeling)기능, 보안모듈(SELinux Module), 감사(Audit)기능을 통해서 프로세스가 실행시 올바른 컨텍스트 레이블(Context Label)를 가지고 있는지 SELinux 모듈를 통해서 확인 후 실행 여부를 결정한다. 이를 통해서 컨테이너에서 올바른 레이블을 가지고 실행한 프로세스는 문제없이 실행이 되나, 레이블이 올바르지 않는 경우 실행을 중지한다.

CRI는 runc를 통해서 컨테이너를 생성해주는 런타임 프로그램이다. CRI는 주요 목적은 runc기반으로 생성되는 가벼운 구조로 빠르게 생성/조회/제거 같은 관리기능을 제공한다. CRI는 독립적으로 사용이 가능하지만, Podman처럼 다양한 기능을 제공하지 않는다.

CRI는 앞서 이야기 하였지만 컨테이너 관리에는 관여하지만 이미지 빌드나 혹은 Docker같은 오케스트레이션 기능은 제공하지 않는다.

CRI는 기본적으로 두 가지 컨테이너 생성기능을 제공하는데 첫 번째는 포드(Pod)생성 그리고 컨테이너(Container)생성 이다. 간단하게 포드와 컨테이너 생성이 가능하지만, Docker처럼 다양하게 네트워크 및 스토리지 그리고 이미지 생성같은 기능을 제공하지 않는다. 또한 CRI는 독립적인 고수준 컨테이너 런타임 환경을 제공하지 않으며, 쿠버네티스나 혹은 오픈시프트 같은 컨테이너 인프라스트럭처 환경에서 최적으로 동작하도록 구성이 되어있다.

Podman
Podman은 CRI를 통해서 동작하는게 아니라 Podman CLI를 통해서 직접적으로 컨테이너를 생성 및 관리를 할 수 있다. 거의 Docker와 동일한 기능을 제공한다.
Podman은 Docker를 대처하기 위한 컨테이너 관리자(Container Management)이다. CRI와 다른 부분은 CRI컨테이너 생성 및 실행을 위한 고레벨 컨테이너 런타임이며, Podman은 CRI와 비슷하지만 Docker처럼 좀 더 많은 부분에 대해서 자원을 관리 및 담당한다.

Podman과 CRI를 비교하였을때 다음과 같은 기능을 CRI에 비해서 풍부하게 제공한다.

컨테이너 중지 및 재시작 기능을 제공한다. CRI는 컨테이너를 재시작하는 기능을 제공하지 않으며 CRI는 컨테이너 중지 후 시작하는 기능만 제공한다
컨테이너 이미지 생성 및 관리기능을 제공한다. CRI는 CRI-T라는 기능을 제공하지만 이미지를 생성하는 하는 기능이 아닌, 이미지를 엔코드(encode), 디코드(decode)기능을 제공한다.

또한 Docker에서 지원하는 거의 대다수의 기능을 그대로 사용할 수 있도록 Podman동일한 기능을 제공하기 때문에 CRI보다 다양한 관리 기능 그리고 컨테이너 운영이 가능하다.

컨테이너 런타임(runc, CRI)와 podman 관계
Podman은 CRI와 라이브러리 및 자원을 공유하지 않으며 각기 독립적인 자원을 가지고 운영이 된다. 또한 Podman은 Docker CLI와 비슷하게 명령어 줄(command line interface)를 제공한다. 이를 통해서 Docker와 호환성을 유지하며, Docker와 마찬가지로 이미지 생성이 가능하다.

Podman에서 생성하는 컨테이너는 CRI를 통해서 생성되지 않으며, 바로 runc를 통해서 컨테이너 생성 및 관리를 한다. Podman은 OCI runtime(runc)를 통해서 컨테이너를 생성하는 제일 기본적인 도구라고 생각하며 된다.

조금 더 쉽게 구분하자면 CRI는 쿠버네티스나 오픈시프트 같은 컨테이너 플랫폼 환경을 운영하는 시스템 관리자(System Administror)에 적절하며, Podman은 일반적으로 개발자가 개발용도로 사용하기에 매우 적절하다.

docker/podman 명령어 비교

podman의 명령어는 기본적으로 docker명령하고 비슷하게 구성이 되어 있다. podman패키지에는 기존 도커와 호환이 되는 명령어 패키지도 있으며, 이 패키지는 RHEL 및 CentOS에 공통적으로 다음과 같은 이름으로 제공이 된다. yum명령어는 혹은 dnf명령어를 통해서 확인이 가능하다.

```
# dnf search podman-docker
# dnf install podman-docker
```

podman에는 pod라는 하위 명령어가 있는데, pod라는 하위 명령어는 컨테이너 생성이 아닌, Pod 컨테이너를 생성하는 명령어 이다. 쿠버네티스에서 사용하는 Pod와 동일한 기능을 구현하며, Pod는 사용자가 생성한 애플리케이션을 Pod에 연결하여 프로그램을 Pod기반으로 동작하도록 해준다. 그 이외 명령어는 동일하게 사용한다.

podman기반으로 서비스 구성하기

podman기반으로 간단하게 서비스를 구성한다. podman은 앞에서 이야기 하였지만, 기본적으로 두 가지 환경으로 제공한다. 첫 번째는 Pod생성 두 번째는 일반적인 컨테이너 런타임 환경이다. 일반적으로 많이 사용하는 컨테이너 런타임 환경으로 구성하는 경우 다음과 같은 명령어로 생성이 가능하다.

```
$ podman run -dt docker.io/library/centos
```

기존 도커와 사용방법은 동일하다. 위의 명령어를 실행하면 도커와 비슷한 방법으로 컨테이너 이미지 및 프로그램 실행을 한다.

1. docker.io에서 centos이미지를 내려받기를 시도한다
2. 이미지가 있는 경우, 1번 과정을 무시하며, 없는 경우 내려받기가 완료가 될때까지 런타임은 대기한다
3. 이미지가 내려받기가 완료가 되면, 런타임을 파일 시스템을 메모리에 불러오면서 런타임 임시 디렉터리에 루트 파일 시스템을 마운트 후, 프로그램을 실행한다

위에서 명시한 명령어 기반으로 컨테이너를 생성/제거/조회 및 관리를 직접 해보도록 한다.

컨테이너 생성

컨테이너 생성 방법은 몇가지 방법이 있다. 첫 번째는 생성 및 실행을 동시에 하는 podman run, 두 번째는 생성만 하는 podman create 명령어가 있다. 제일 많이 사용하는 centos7컨테이너 이미지 기반으로 생성하도록 하겠다.

이미지를 내려받기 위해서 다음 명령어로 내려받기를 한다.

```
$ podman pull centos:7
$ podman pull centos
```

podman기반으로 이미지를 내려받기 하면 기본 저장소가 registry.redhat.io, registry.access.redhat.com으로 되어 있다. 그래서 위의 2개의 주소를 먼저 시도한 후, 그 다음에 docker.io에 내려받기를 시도한다. 이미지 내려받기가 완료가 되면 다음 명령어로 이미지 목록을 확인한다.

```
$ podman images
REPOSITORY              TAG     IMAGE ID        CREATED         SIZE
docker.io/library/centos   7            7e6257c9f8d8   5 days ago   211 MB
docker.io/library/centos   latest   0d120b6ccaa8   5 days ago   222 MB
```

처음에 받은 centos 7이미지 및 나머지 하나는 latest라는 이름으로 출력이 된다. 이미지의 버전을 자세하게 확인한다. 두개의 이미지는 이미지 아이디는 다르지만 같은 이미지이다. 일단, 이미지에 대한 이야기는 좀 더 뒤에서 진행하고, 실제로 직접 컨테이너를 생성하고 실행 해보도록 한다.

```
$ podman create --name test1 centos:7
220a56536aaad418329a81e04495d9a79d79e58176b54cfbe23e6f7792a725d8
$ podman run --name test2 centos:7
```

두개의 컨테이너를 create 및 run명령어로 생성하였으며, --name이라는 옵션으로 test1,2라는 이름을 각각 정의했다. 실행이 완료가 되면, 다음과 같은 명령어로 컨테이너 목록을 확인한다.

```
$ podman ps -a
CONTAINER ID  IMAGE                      COMMAND CREATED         STATUS                  PORTS  NAMES
9734c3d109fd  docker.io/library/centos:7  /bin/bash  5 seconds ago   Exited (0) 4 seconds ago
test2
220a56536aaa  docker.io/library/centos:7  /bin/bash  16 seconds ago  Created
test1
```

podman ps명령어로는 컨테이너 목록이 보이지 않는다. 그래서 -a라는 옵션을 통해서 중지된 컨테이너까지 조회를 하도록 한다. 컨테이너 생성은 완료가 되었지만 프로그램이 실행이 되지 않았기 때문에 런타임은 이미지를 메모리에 상주 후 바로 종료하도록 한다. 그래서, 다음과 같은 명령어로 간단하게 프로그램을 실행하여 컨테이너 실행 상태를 유지하도록 한다.

```
$ podman run -it centos /bin/bash
```

centos이미지 기반으로 컨테이너를 생성하면서 bash스크립트를 실행한다. 올바르게 컨테이너 프로세서가 실행이 되면, 컨테이너가 가지고 있는 bash를 실행하면서 컨테이너 쉘을 실행한다.

```
$ podman run -it centos /bin/bash
[root@2214edf970af /]#
```

컨테이너 하지만 해당 컨테이너는 bash를 종료하면, 런타임은 즉시 해당 컨테이너 실행을 종료한다.

```
[root@2214edf970af /]# exit
exit
$ podman ps
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
$
```

종료하면 더 이상 런타임은 해당 bash프로세스를 종료한다. 그러므로 지속적으로 실행하기 위해서는 지속적으로 동작이 가능한 프로세스 하나를 실행해야 한다.

```
$ podman run -dit --name bg-sleep centos /bin/sleep 10000
9007c41da272532ac0d0f3dcb7d3cafd6c4ad7d5933522394a39c24ebcfb4557
$ podman ps
CONTAINER ID  IMAGE                              COMMAND          CREATED          STATUS          P O R T S
NAMES
9007c41da272  docker.io/library/centos:latest  /bin/sleep 10000  9 seconds ago  Up 9 seconds ago
bg-sleep
$
```

sleep명령어로 간단하게 하나 실행한다. run옵션에 -d -i, -t옵션을 사용하였다. -d 옵션은 detach의 약자이며, 실행하는 즉시 런타임은 백그라운드로 실행한다. -i는 interactive약자이며 대화 STDIN를 열어둔다. -t옵션은 tty약자이며 가짜 터미널(pseudo-TTY)를 실행하여 터미널 프로그램 실행이 가능하도록 한다.

결론적으로, 위의 컨테이너는 10000초 동안 컨테이너가 실행이 되며, 이 시간이 지나가면 해당 컨테이너는 즉시 종료한다.

podman은 수 많은 명령어는 여러 기능을 제공하고 있으며 다음 과정을 위해 준비를 해 두어야 한다.

컨테이너 이미지 생성 및 빌드

컨테이너 이미지 구성 및 관리에 대해서 설명하도록한다. Podman은 기존 도커에서 사용하는 Dockerfile를 가지고 그대로 사용이 가능하다. 기본적으로 사용하는 Dockerfile 명령어에 대해서 확인하자.

도커는 Docker Instruction이라는 별도의 명령어 구조를 가지고 있다. 이 도커 명령어 구조는 순차적으로 처리하는 구조이며 최종적으로 사용자가 원하는 프로그램을 실행하는 구조를 가지고 있다. 도커 명령어는 다음 명령어를 제일 많이 사용한다.

명령어 설명

FROM 빌드시 사용할 이미지

WORKDIR 빌드시 사용할 작업 디렉토리를 변경

COPY 파일이나 혹은 디렉토리를 복사시 사용

ADD 원격에서 파일을 내려받기 하거나 혹은 복사시 사용

RUN 특정 명령어를 일시적으로 실행

EXPOSE 컨테이너 실행시 특정 포트를 호스트에 노출한다

CMD 컨터이너 실행시 특정 프로그램을 실행 합니다. 이 명령어는 RUN과 다르게 계층을 생성하지 않습니다.

일반적으로 많이 사용하는 Dockerfile 명령어이다. 이 명령어는 다음과 같은 규칙으로 작성한다

```
/home/studnet/test/Dockerfile
FROM centos:latest
COPY dummy.img /tmp
ADD http://localhost/dummy2.img /tmp
RUN echo “Hello Dockerfile!”
EXPOSE 8080
CMD [ “sleep 1000” ]
```

위의 파일은 /home/student에 test/라는 디렉토리를 생성하고, Dockerfile파일을 생성 후, 다음과 같은 동작을 수행한다

1. FROM은 이미지 레지스트리 서버에서 centos마지막 이미지를 내려받기한다. 만약, 이미지가 없는 경우
2. COPY는 컨테이너 이미지에 dummy.img라는 파일을 컨테이너 내부 디렉토리인 /tmp에 복사한다
3. ADD는 현재 리눅스 머신에 설치가 되어 있는 웹 서버에서 dummy2.img이미지를 복사 후 /tmp에 복사를 한다
4. RUN명령어를 통해서 컨테이너에서 “Hello Dockerfile!”라는 메세지를 출력한다
5. EXPOSE명령어를 통해서는 의미는 없지만, 포트 8080/TCP를 열어둔다
6. 최종적으로 컨테이너가 메모리에 로드 후 실행하는 프로그램은 “sleep 1000”이라는 명령어를 실행한다

이미지가 빌드가 되면, 다음과 같은 명령어로 이미지를 podman에서 확인이 가능하다.

```
$ podman images
REPOSITORY                    TAG      IMAGE ID        CREATED          SIZE
<none>                        <none>   0a61be5d627c    34 seconds ago   224 MB
```

이미지는 생성하였지만, 위의 내용처럼 저장소(Repository) 및 인식표(Tag)가 표시가 되어있지 않다. 이 경우에는 다음과 같은 명령어로 이미지를 Tag가 가능하다.

```
$ podman tag 0a61be5d627c localhost:5000/podman/hello
$ podman images
REPOSITORY                    TAG      IMAGE ID        CREATED          SIZE
localhost:5000/podman/hello   latest   0a61be5d627c    9 minutes ago    224 MB
docker.io/library/centos      latest   0d120b6ccaa8    6 days ago       222 MB
quay.io/libpod/alpine_nginx   latest   3ef70f7291f4    21 months ago    9.21 MB
k8s.gcr.io/pause              3.1      da86e6ba6ca1    2 years ago      749 kB
```

tag명령어로 <none>으로 표시가 되어있던 이미지를 localhost:5000서버에 있다고 저장소 위치를 표시하였으며, 인식표 역시 latest로 표시가 되었다. 하지만, 이 이미지는 아직 localhost:5000의 이미지 레지스트리 서버에 저장이 되어

있지 않다. 이 부분은 좀 더 뒤에서 이야기를 하고 추가적으로 몇가지 명령어를 더 실행 후 좀 더 복잡한 이미지를 구성하도록 한다.

앞에서 우리는 Dockerfile기반으로 기본 이미지(base image)인 centos에 몇가지 명령어를 수행하였다. 이 명령어가 올바르게 수행이 되었는지 확인하기 위해서는 제일 쉬운 방법은 history명령어로 확인하는 방법이다.

```
$ podman history 0a61be5d627c
ID              CREATED         CREATED BY                                      SIZE    COMMENT
0a61be5d627c    12 minutes ago  /bin/sh -c #(nop) CMD ["sleep 10000"]           0B
<missing>       12 minutes ago   /bin/sh -c #(nop) EXPOSE 8080                  0B
8f93df970149    12 minutes ago  /bin/sh -c echo "Hello Dockerfile"              2.56kB
a08e11f8656f    12 minutes ago  /bin/sh -c #(nop) ADD 30e14955ebf1352266dc...   1.051MB
539c8b0abdf7    12 minutes ago  /bin/sh -c #(nop) COPY file:6a8ce3b8e34ead...   1.051MB
0d120b6ccaa8    6 days ago      /bin/sh -c #(nop) CMD ["/bin/bash"]             1.051MB
<missing>       6 days ago      /bin/sh -c #(nop) LABEL org.label-schema.s...   1.051MB
<missing>       6 days ago      /bin/sh -c #(nop) ADD file:538afc0c5c964ce...   222.4MB
```

history명령어로 이미지를 조회하면, 이미지 생성시 사용한 Dockerfile명령어와 그리고 실행시 발생한 데이터의 크기가 표시가 된다. 우리가 만든 이미지를 런타임에서 임시적으로 실행하도록 한다.

```
$ podman run -d --name test1 0a61be5d627c
```

위의 명령어로 임시적으로 컨테이너를 실행하면, 컨테이너는 실행하자마자 바로 종료하게 된다. 그 이유는 컨테이너에서 실행하는 프로그램은 명확하게 sleep 1000으로 시작이 되어야 하지만, cmd 혹은 entrypoint는 명령어 호출 후 바로 종료하기 때문에 결론적으로 프로그램이 올바르게 실행이 되지 않는다. 그래서, 이러한 문제를 방지하기 위해서 몇 가지 기능을 더 추가 후 컨테이너 이미지를 실행 하도록 한다.

```
$vi Dockerfile
FROM centos:latest
COPY dummy.img /tmp
ADD http://localhost/dummy2.img /tmp
RUN echo "Hello Dockerfile"
RUN mkdir /apps
ADD run.sh /apps/run.sh
RUN chmod +x /apps/run.sh
EXPOSE 8080
CMD ["/apps/run.sh"]
```

Dockerfile에 굵게표시가 되어있는 부분을 추가 하였으며 기능은 다음과 같다

1. RUN mkdir /apps는 컨테이너 내부에 /apps라는 디렉토리를 생성한다
2. ADD run.sh라는 파일을 /apps/밑으로 run.sh를 복사한다
3. RUN /apps/run.sh에 있는 run.sh파일에 실행권한을 chmod +x로 할당한다
4. CMD는 /app/run.sh라는 파일을 컨테이너 실행시 실행한다

이미지 빌드를 시작한다. 이미지 빌드를 위해서 podman build를 사용하는데, 이번에는 -t옵션을 사용하여 이미지 빌드가 완료가 되면, 저장소 및 인식표를 구성후 podman 런타임에 업로드한다.

```
# podman build -t localhost:5000/podman:v1 .
STEP 1: FROM centos:latest
STEP 2: COPY dummy.img /tmp
--> Using cache 539c8b0abdf7a26b08943e55873e8b871bdcede79f2e69e4064833c1c5140599
STEP 3: ADD http://localhost/dummy2.img /tmp
--> Using cache a08e11f8656f6861c5bc77036d03a40fac715a5b1c63fe620e3395a77d143220
STEP 4: RUN echo "Hello Dockerfile"
--> Using cache 8f93df970149f0d6a2446f23661f2f15ec10cfb3fef835e2144e1b094575f9bf
STEP 5: RUN mkdir /apps
--> Using cache 180e0918acc36bf7cc39c9168c1fc081901220cb4a80010345b53b4cde3e5930
STEP 6: ADD run.sh /apps/run.sh
--> Using cache e029a4ed3a91c72a51b0530d8f8b98fc53cee9632a1111ddb21d72abe3884252
```

```
STEP 7: RUN chmod +x /apps/run.sh
--> Using cache ce0e7b8f647f94464f88e717daaa96de738981e85c7cbcab97043f797426dd51
STEP 8: EXPOSE 8080
--> Using cache de9e565ea59e5411a32dc60c84af31195003fd1f92129403eefe51ddb5e13bf6
STEP 9: CMD ["/apps/run.sh"]
--> Using cache
593d8e0ed84f25539a82fbe5fa6c871ace07b87ba4e8295337648bf2ab170aa6
STEP 10: COMMIT localhost:5000/podman:v1
593d8e0ed84f25539a82fbe5fa6c871ace07b87ba4e8295337648bf2ab170aa6
```

빌드가 완료가 되면 images명령어로 올바르게 이미지가 podman에 등록이 되었는지 확인한다.

```
$ podman images
REPOSITORY                    TAG      IMAGE ID        CREATED         SIZE
localhost:5000/podman         v1       593d8e0ed84f    10 minutes ago  224 MB
```

올바르게 런타임이 등록이 되었으면, run명령어로 컨테이너를 실행한다. 실행시 다음과 같은 명령어로 컨테이너를 실행한다. 환경마다 다르지만 일반적으로 다음과 같은 메세지가 출력이 되면서 컨테이너가 실행이 된다.

```
$ podman run -d --name sleep-v2 593d8e0ed84f
[ 2490.666230] IPv6: ADDRCONF(NETDEV_UP): eth0: link is not ready
[ 2490.673095] IPv6: ADDRCONF(NETDEV_UP): veth984dab02: link is not ready
[ 2490.674830] IPv6: ADDRCONF(NETDEV_CHANGE): veth984dab02: link becomes ready
[ 2490.676476] IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
[ 2490.678928] cni-podman0: port 3(veth984dab02) entered blocking state
[ 2490.680490] cni-podman0: port 3(veth984dab02) entered disabled state
[ 2490.682193] device veth984dab02 entered promiscuous mode
[ 2490.683530] cni-podman0: port 3(veth984dab02) entered blocking state
[ 2490.685016] cni-podman0: port 3(veth984dab02) entered forwarding state
[ 2490.765779] SELinux: mount invalid.  Same superblock, different security settings for (dev mqueue,
type mqueue)
4a58b019732a194c918c161a82c73ec48b5a429e353193ae9f83a884a030a1f0
```

실행이 완료되면 ps명령어로 컨테이너 프로세스가 올바르게 메모리에 상주하여 동작하는지 확인한다.

```
# podman ps
CONTAINER ID  IMAGE                           COMMAND         CREATED                S T A T U S
        PORTS  NAMES
4a58b019732a  localhost:5000/podman:v1        /apps/run.sh  About a minute ago  Up About a minute ago
        sleep-v2
```